Numerical Linear Algebra II: Lecture Notes

Eigenvalues, SVD, PCA, PageRank, Sparse Reorderings, and Krylov Subspace Methods

Jakub Kružík
jakub.kruzik@vsb.cz

2026

Contents

1 Eigenvalues, Eigenvectors, and Matrix Diagonalization **2**

1.1 Definition and Characteristic Polynomial 2

1.2 Trace, Determinant, and Basic Spectral Properties 2

1.3 Matrix Diagonalization 4

1.4 Applications to Difference Equations and Markov Chains 5

1.5 Applications to Systems of Differential Equations 8

2 Singular Value Decomposition (SVD) **10**

2.1 Geometric Motivation and Derivation 10

2.2 Key Properties and Eckart–Young–Mirsky Theorem 11

2.3 Computing the SVD via Golub–Kahan Bidiagonalization 11

3 Principal Component Analysis (PCA) **12**

3.1 Data Matrix and Basic Statistics 12

3.2 Centering the Data and the Covariance Matrix 13

3.3 PCA via Eigen-Decomposition and SVD 13

3.4 Total Variance and Explained Variance Ratio 14

4 PageRank Algorithm **16**

4.1 Introduction, Network Centrality, and Link Democracy 16

4.2 Transition Matrix, Random Walk $\boldsymbol{p}(t)$, and the Dominant Eigenvector $\boldsymbol{r}$ 18

4.3 Pathologies of Pure Link-Following and Teleportation (Damping Factor) 19

4.4 Power Iteration Algorithm and Handling Sink Nodes 20

4.5 The Google Matrix, Dominant Eigenvector Theory, and Personalized PageRank . 22

5 Sparse Direct Solvers and Matrix Reorderings **23**

5.1 Block LU Factorization, Fill-In, and Elimination Graphs 23
5.2 Reordering Algorithms 25

6 Quadratic Function Minimization and GMRES **26**
6.1 Quadratic Cost Function and Level Sets 26
6.2 Krylov Subspace Minimization and the GMRES Idea 27
6.3 Arnoldi Orthogonalization and the Hessenberg Decomposition 28
6.4 Reducing GMRES to a Small Least-Squares Problem 28

---

1 Eigenvalues, Eigenvectors, and Matrix Diagonalization

1.1 Definition and Characteristic Polynomial

Definition 1.1 (Eigenvalues and Eigenvectors)**.** Let $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ be a square matrix. A non-zero vector $\boldsymbol{v} \in \mathbb{C}^n$ ($\boldsymbol{v} \neq \boldsymbol{0}$) is called an *eigenvector* of $\boldsymbol{A}$, and $\lambda \in \mathbb{C}$ is its corresponding *eigenvalue*, if

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}. \tag{1}$$

Geometrically, multiplying an eigenvector $\boldsymbol{v}$ by the matrix $\boldsymbol{A}$ does not change its direction (line of action); it only scales the vector by the factor λ.

Rewriting (1) as a homogeneous linear system gives

$$(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{v} = \boldsymbol{0}.$$

Since we require a non-trivial solution $\boldsymbol{v} \neq \boldsymbol{0}$, the matrix $\boldsymbol{A}-\lambda\boldsymbol{I}$ must be singular. Consequently, λ is an eigenvalue of $\boldsymbol{A}$ if and only if it is a root of the *characteristic polynomial*:

$$p(\lambda) = \det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0. \tag{2}$$

1.2 Trace, Determinant, and Basic Spectral Properties

The eigenvalues of $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ are closely related to its matrix invariants:

- **Trace:** The sum of the diagonal entries equals the sum of the eigenvalues:

$$\operatorname{tr}(\boldsymbol{A}) = \sum_{i=1}^{n} a_{ii} = \sum_{i=1}^{n} \lambda_i.$$

- **Determinant:** The determinant equals the product of the eigenvalues:

$$\det(\boldsymbol{A}) = \prod_{i=1}^{n} \lambda_i.$$

  For a 2×2 matrix, expanding $\det(\boldsymbol{A} - \lambda\boldsymbol{I})$ yields $\lambda^2 - \operatorname{tr}(\boldsymbol{A})\lambda + \det(\boldsymbol{A}) = 0$. When computing eigenvalues by hand, however, writing out the standard determinant $\det(\boldsymbol{A} - \lambda\boldsymbol{I}) = 0$ directly is generally preferable (as it often preserves factored terms such as $(a-\lambda)^2 - b^2$), while $\sum \lambda_i = \operatorname{tr}(\boldsymbol{A})$ and $\prod \lambda_i = \det(\boldsymbol{A})$ serve as a quick verification check.

- **Spectral Shift:** Adding $\alpha \boldsymbol{I}$ ($\alpha \in \mathbb{R}$) shifts all eigenvalues by α while preserving the eigenvectors:
$$(\boldsymbol{A} + \alpha \boldsymbol{I})\boldsymbol{v} = \boldsymbol{A}\boldsymbol{v} + \alpha \boldsymbol{I}\boldsymbol{v} = \lambda \boldsymbol{v} + \alpha \boldsymbol{v} = (\lambda + \alpha)\boldsymbol{v}.$$
- **Inverse Matrix:** If $\boldsymbol{A}$ is invertible (i.e., $\lambda_i \neq 0$ for all i), multiplying $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$ by $\boldsymbol{A}^{-1}$ and dividing by λ yields
$$\boldsymbol{v} = \lambda \boldsymbol{A}^{-1}\boldsymbol{v} \quad \Longrightarrow \quad \boldsymbol{A}^{-1}\boldsymbol{v} = \frac{1}{\lambda}\boldsymbol{v}.$$
Thus, $\boldsymbol{A}^{-1}$ has the same eigenvectors as $\boldsymbol{A}$ with eigenvalues $1/\lambda_i$.
- **Matrix Powers:** Multiplying $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$ repeatedly by $\boldsymbol{A}$ gives
$$\boldsymbol{A}^2\boldsymbol{v} = \boldsymbol{A}(\boldsymbol{A}\boldsymbol{v}) = \lambda \boldsymbol{A}\boldsymbol{v} = \lambda^2 \boldsymbol{v}, \quad \text{and in general,} \quad \boldsymbol{A}^k\boldsymbol{v} = \lambda^k \boldsymbol{v}.$$
- **Complex Conjugate Pairs for Real Matrices:** For *any* real matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, the characteristic polynomial $p(\lambda) = \det(\boldsymbol{A} - \lambda \boldsymbol{I})$ has real coefficients, so any non-real eigenvalues must occur in **complex conjugate pairs** λ and $\overline{\lambda}$. Equivalently, taking the complex conjugate of $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$ and using $\overline{\boldsymbol{A}} = \boldsymbol{A}$ gives
$$\overline{\boldsymbol{A}\boldsymbol{v}} = \overline{\lambda \boldsymbol{v}} \quad \Longrightarrow \quad \boldsymbol{A}\overline{\boldsymbol{v}} = \overline{\lambda}\,\overline{\boldsymbol{v}},$$
showing that if $(\lambda, \boldsymbol{v})$ is an eigenpair of a real matrix $\boldsymbol{A}$, then $(\overline{\lambda}, \overline{\boldsymbol{v}})$ is also an eigenpair.
- **Symmetry and Skew-Symmetry:**
  - If $\boldsymbol{A}$ is real and **symmetric** ($\boldsymbol{A} = \boldsymbol{A}^T \in \mathbb{R}^{n\times n}$), then all eigenvalues are real ($\lambda_i \in \mathbb{R}$) and there exists a full set of real, mutually orthogonal eigenvectors ($\boldsymbol{v}_i \in \mathbb{R}^n$).
  - If $\boldsymbol{Q}$ is real and **skew-symmetric** (anti-symmetric, $\boldsymbol{Q}^T = -\boldsymbol{Q}$), its eigenvalues are purely imaginary (or zero) and, since $\boldsymbol{Q}$ is real, occur in complex conjugate pairs $\pm i\omega$.

Example 1.1 (Permutation/Reflection Matrix)**.** Consider the symmetric matrix
$$\boldsymbol{A} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$
Computing the characteristic polynomial via $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$:
$$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = \begin{vmatrix} -\lambda & 1 \\ 1 & -\lambda \end{vmatrix} = \lambda^2 - 1 = (\lambda - 1)(\lambda + 1) = 0 \quad \Longrightarrow \quad \lambda_1 = 1, \quad \lambda_2 = -1,$$
with corresponding eigenvectors:
$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \lambda_1 = 1; \qquad \boldsymbol{v}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \quad \lambda_2 = -1.$$
As a check, $\lambda_1 + \lambda_2 = 1 + (-1) = 0 = \operatorname{tr}(\boldsymbol{A})$ and $\lambda_1\lambda_2 = -1 = \det(\boldsymbol{A})$.

Example 1.2 (Shifted Symmetric Matrix)**.** Consider the matrix obtained by adding $3\boldsymbol{I}$ to the previous example:
$$\boldsymbol{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$
The characteristic polynomial is
$$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = \begin{vmatrix} 3 - \lambda & 1 \\ 1 & 3 - \lambda \end{vmatrix} = (3 - \lambda)^2 - 1 = \lambda^2 - \underbrace{6}_{\operatorname{tr}(\boldsymbol{A})}\lambda + \underbrace{8}_{\det(\boldsymbol{A})} = (\lambda - 4)(\lambda - 2) = 0.$$
Thus $\lambda_1 = 4$ and $\lambda_2 = 2$ (which are indeed $1 + 3$ and $-1 + 3$ from the shift property $\boldsymbol{A} + 3\boldsymbol{I}$). Solving $(\boldsymbol{A} - \lambda_i \boldsymbol{I})\boldsymbol{v}_i = \boldsymbol{0}$:

- For $\lambda_1 = 4$: $\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \mathbf{0} \implies \boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.
- For $\lambda_2 = 2$: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} = \mathbf{0} \implies \boldsymbol{v}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$.

Example 1.3 (Rotation / Skew-Symmetric Matrix)**.** Consider the 90° counter-clockwise rotation matrix in $\mathbb{R}^2$, which maps $\boldsymbol{x} = (1, 0)^T$ to $\boldsymbol{Q}\boldsymbol{x} = (0, 1)^T$:

$$\boldsymbol{Q} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Since $\boldsymbol{Q}$ rotates every non-zero real vector by 90°, no real vector stays on its own line. Indeed, solving the characteristic equation gives

$$\det(\boldsymbol{Q} - \lambda \boldsymbol{I}) = \begin{vmatrix} -\lambda & -1 \\ 1 & -\lambda \end{vmatrix} = \lambda^2 + 1 = 0 \quad \implies \quad \lambda_1 = i, \quad \lambda_2 = -i,$$

which are complex conjugate purely imaginary numbers (satisfying $\lambda_1 + \lambda_2 = 0 = \operatorname{tr}(\boldsymbol{Q})$ and $\lambda_1 \lambda_2 = 1 = \det(\boldsymbol{Q})$).

1.3 Matrix Diagonalization

Suppose $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ has n linearly independent eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ with corresponding eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$. Placing the eigenvectors as columns into a matrix $\boldsymbol{Q} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n)$ gives

$$\boldsymbol{A}\boldsymbol{Q} = \boldsymbol{A}(\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n) = (\lambda_1 \boldsymbol{v}_1, \lambda_2 \boldsymbol{v}_2, \ldots, \lambda_n \boldsymbol{v}_n) = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n) \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix} = \boldsymbol{Q}\boldsymbol{\Lambda},$$

where $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$. Since the columns of $\boldsymbol{Q}$ are linearly independent, $\boldsymbol{Q}$ is invertible, yielding the *eigenvalue decomposition (diagonalization)*:

$$\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{-1}. \tag{3}$$

If $\boldsymbol{A}$ is symmetric ($\boldsymbol{A} = \boldsymbol{A}^T$), the eigenvectors can be chosen orthonormal, so $\boldsymbol{Q}$ is orthogonal ($\boldsymbol{Q}^{-1} = \boldsymbol{Q}^T$) and $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$.

Proposition 1.1 (Sufficient Condition for Diagonalizability)**.** *If all eigenvalues of $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ are distinct ($\lambda_i \neq \lambda_j$ for all $i \neq j$), then the corresponding eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ are linearly independent, and $\boldsymbol{A}$ is diagonalizable.*

Remark 1.1 (Defective Matrices). Not every matrix is diagonalizable. For example, the upper triangular matrix

$$\boldsymbol{A} = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$$

has a repeated eigenvalue $\lambda_1 = \lambda_2 = 3$, but solving $(\boldsymbol{A} - 3\boldsymbol{I})\boldsymbol{v} = \mathbf{0}$ yields only a single independent eigenvector direction $\boldsymbol{v} = (1, 0)^T$. Such a matrix lacks n linearly independent eigenvectors and cannot be diagonalized.

Remark 1.2 (Repeated Eigenvalues and the Identity Matrix). Having repeated eigenvalues does not prevent a matrix from being diagonalizable; distinct eigenvalues are a sufficient condition, not a necessary one. For instance, the identity matrix

$$\boldsymbol{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

has the repeated eigenvalue $\lambda_1 = \lambda_2 = 1$, yet every non-zero vector $\boldsymbol{v} \in \mathbb{R}^2$ is an eigenvector ($\boldsymbol{I}\boldsymbol{v} = 1 \cdot \boldsymbol{v}$). Consequently, $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ is diagonalizable by *any* invertible matrix $\boldsymbol{Q}$, since $\boldsymbol{Q}\boldsymbol{I}\boldsymbol{Q}^{-1} = \boldsymbol{I}$.

Remark 1.3 (Algebraic vs. Geometric Multiplicity). The difference between the two examples above is captured by the two notions of multiplicity for an eigenvalue λ:

- **Algebraic multiplicity** $m_a(\lambda)$: the multiplicity of λ as a root of the characteristic polynomial $p(\lambda) = \det(\boldsymbol{A} - \lambda \boldsymbol{I})$.
- **Geometric multiplicity** $m_g(\lambda)$: the dimension of the eigenspace $\mathcal{N}(\boldsymbol{A} - \lambda \boldsymbol{I})$, i.e., the number of linearly independent eigenvectors associated with λ.

For every eigenvalue λ, these multiplicities satisfy

$$1 \leq m_g(\lambda) \leq m_a(\lambda).$$

A matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ is diagonalizable (over $\mathbb{C}$) **if and only if** $m_g(\lambda) = m_a(\lambda)$ for every eigenvalue λ:

- For $\boldsymbol{A} = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$, $\lambda = 3$ has $m_a(3) = 2$ but $m_g(3) = 1 < m_a(3)$, so $\boldsymbol{A}$ is *defective* (not diagonalizable).
- For $\boldsymbol{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $\lambda = 1$ has $m_a(1) = 2$ and $m_g(1) = 2 = m_a(1)$, so $\boldsymbol{I}$ is diagonalizable.

Using diagonalization (3), computing matrix powers becomes trivial:

$$\boldsymbol{A}^2 = (\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{-1})(\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{-1}) = \boldsymbol{Q}\boldsymbol{\Lambda}^2\boldsymbol{Q}^{-1}, \qquad \boldsymbol{A}^k = \boldsymbol{Q}\boldsymbol{\Lambda}^k\boldsymbol{Q}^{-1},$$

where $\boldsymbol{\Lambda}^k = \operatorname{diag}(\lambda_1^k, \ldots, \lambda_n^k)$.

1.4 Applications to Difference Equations and Markov Chains

Consider a linear discrete-time dynamical system (difference equation):

$$\boldsymbol{u}_{k+1} = \boldsymbol{A}\boldsymbol{u}_k \quad \Longrightarrow \quad \boldsymbol{u}_1 = \boldsymbol{A}\boldsymbol{u}_0, \quad \boldsymbol{u}_2 = \boldsymbol{A}\boldsymbol{u}_1 = \boldsymbol{A}^2\boldsymbol{u}_0, \quad \ldots, \quad \boldsymbol{u}_k = \boldsymbol{A}^k\boldsymbol{u}_0.$$

If $\boldsymbol{A}$ has linearly independent eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$, we can expand the initial state $\boldsymbol{u}_0$ in the eigenvector basis:

$$\boldsymbol{u}_0 = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n = \boldsymbol{Q}\boldsymbol{c}, \quad \text{where } \boldsymbol{c} = (c_1, c_2, \ldots, c_n)^T = \boldsymbol{Q}^{-1}\boldsymbol{u}_0.$$

Applying $\boldsymbol{A}^k$ then gives an explicit closed-form formula for any step k:

$$\boldsymbol{u}_k = \boldsymbol{A}^k\boldsymbol{u}_0 = c_1\lambda_1^k\boldsymbol{v}_1 + c_2\lambda_2^k\boldsymbol{v}_2 + \cdots + c_n\lambda_n^k\boldsymbol{v}_n = \boldsymbol{Q}\boldsymbol{\Lambda}^k\boldsymbol{c}. \tag{4}$$

Stability of Difference Equations. From (4), the long-term behavior of $\boldsymbol{u}_k$ as $k \to \infty$ is governed entirely by the magnitudes $|\lambda_i|$ of the eigenvalues of $\boldsymbol{A}$:

- **Stable (Asymptotically Stable):** $|\lambda_i| < 1$ for all $i = 1, \ldots, n$. Every term $\lambda_i^k \to 0$ as $k \to \infty$, so $\boldsymbol{u}_k \to \boldsymbol{0}$ for every initial state $\boldsymbol{u}_0$.

- **Neutrally Stable:** $|\lambda_i| \leq 1$ for all i, with at least one eigenvalue satisfying $|\lambda_i| = 1$ (and having linearly independent eigenvectors). Modes with $|\lambda_i| < 1$ decay to zero, while modes with $|\lambda_i| = 1$ neither grow nor decay — in particular, if $\lambda_1 = 1$ and $|\lambda_i| < 1$ for $i \geq 2$, the system converges to a non-zero **steady state** $\boldsymbol{u}_\infty = c_1\boldsymbol{v}_1$ (as in Markov chains below).
- **Unstable:** $|\lambda_i| > 1$ for at least one i. The corresponding mode λ_i^k grows exponentially in magnitude as $k \to \infty$, so $\|\boldsymbol{u}_k\| \to \infty$ (whenever $c_i \neq 0$, as in the Fibonacci sequence below).

Example 1.4 (Unstable Difference Equation: Binet's Formula for the Fibonacci Sequence)**.** Consider the Fibonacci sequence $0, 1, 1, 2, 3, 5, 8, 13, \ldots$ defined by $F_0 = 0$, $F_1 = 1$, and the second-order recurrence

$$F_{k+2} = F_{k+1} + F_k.$$

To write this as a first-order system $\boldsymbol{u}_{k+1} = \boldsymbol{A}\boldsymbol{u}_k$, define the state vector $\boldsymbol{u}_k = \begin{pmatrix} F_{k+1} \\ F_k \end{pmatrix}$ with initial state $\boldsymbol{u}_0 = \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$:

$$\begin{cases} F_{k+2} = F_{k+1} + F_k \\ F_{k+1} = F_{k+1} \end{cases} \iff \boldsymbol{u}_{k+1} = \begin{pmatrix} F_{k+2} \\ F_{k+1} \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} F_{k+1} \\ F_k \end{pmatrix} = \boldsymbol{A}\boldsymbol{u}_k.$$

1. **Eigenvalues:**

$$\det(\boldsymbol{A} - \lambda\boldsymbol{I}) = \begin{vmatrix} 1-\lambda & 1 \\ 1 & -\lambda \end{vmatrix} = \lambda^2 - \lambda - 1 = 0 \implies \lambda_{1,2} = \frac{1 \pm \sqrt{1+4}}{2} = \frac{1 \pm \sqrt{5}}{2}.$$

   Thus $\lambda_1 = \frac{1+\sqrt{5}}{2} \approx 1.618 > 1$ (the golden ratio, an unstable growing mode) and $\lambda_2 = \frac{1-\sqrt{5}}{2} \approx -0.618$ ($|\lambda_2| < 1$, a decaying mode). Since $\lambda_1 \neq \lambda_2$, $\boldsymbol{A}$ is diagonalizable.

2. **Eigenvectors:** From $(\boldsymbol{A} - \lambda\boldsymbol{I})\boldsymbol{v} = \begin{pmatrix} 1-\lambda & 1 \\ 1 & -\lambda \end{pmatrix}\boldsymbol{v} = \boldsymbol{0}$, the second row gives $v_1 - \lambda v_2 = 0$, so we can choose

$$\boldsymbol{v}_1 = \begin{pmatrix} \lambda_1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{v}_2 = \begin{pmatrix} \lambda_2 \\ 1 \end{pmatrix}.$$

3. **Initial Condition Coefficients:** We solve $\boldsymbol{u}_0 = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$:

$$\left(\begin{array}{cc|c} \lambda_1 & \lambda_2 & 1 \\ 1 & 1 & 0 \end{array}\right) \sim \left(\begin{array}{cc|c} 1 & 1 & 0 \\ \lambda_1 & \lambda_2 & 1 \end{array}\right) \sim \left(\begin{array}{cc|c} 1 & 1 & 0 \\ 0 & \lambda_2 - \lambda_1 & 1 \end{array}\right).$$

   Since $\lambda_2 - \lambda_1 = \frac{1-\sqrt{5}}{2} - \frac{1+\sqrt{5}}{2} = -\sqrt{5}$, we get

$$c_2(-\sqrt{5}) = 1 \implies c_2 = -\frac{1}{\sqrt{5}}, \qquad c_1 = -c_2 = \frac{1}{\sqrt{5}}.$$

4. **Closed-Form Solution:** Substituting into $\boldsymbol{u}_k = \boldsymbol{A}^k\boldsymbol{u}_0 = c_1\lambda_1^k\boldsymbol{v}_1 + c_2\lambda_2^k\boldsymbol{v}_2$:

$$\begin{pmatrix} F_{k+1} \\ F_k \end{pmatrix} = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^k \begin{pmatrix} \lambda_1 \\ 1 \end{pmatrix} - \left(\frac{1-\sqrt{5}}{2}\right)^k \begin{pmatrix} \lambda_2 \\ 1 \end{pmatrix}\right].$$

Reading off the second component yields *Binet's formula*:

$$\boxed{F_k = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^k - \left(\frac{1-\sqrt{5}}{2}\right)^k\right].}$$

For large k (e.g., F_{100}), since $|\lambda_2| \approx 0.618 < 1$, the second term vanishes and $F_k \approx \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^k$.

Markov Matrices and Steady States. An important class of **neutrally stable** difference equations $\boldsymbol{v}_{t+1} = \boldsymbol{A}\boldsymbol{v}_t$ arises from Markov chains, where $\boldsymbol{A}$ describes state transition probabilities.

Definition 1.2 (Markov / Column-Stochastic Matrix)**.** A square matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ is called a *Markov matrix* (or *column-stochastic matrix*) if:

1. All entries are non-negative: $a_{ij} \geq 0$ for all i, j.
2. Every column sums to 1: $\sum_{i=1}^{n} a_{ij} = 1$ for all $j = 1, \ldots, n$ (equivalently, $\boldsymbol{e}^T\boldsymbol{A} = \boldsymbol{e}^T$ where $\boldsymbol{e} = (1, \ldots, 1)^T$).

Key spectral properties of a Markov matrix $\boldsymbol{A}$:

- **Conservation of total probability/population:** Because $\boldsymbol{e}^T\boldsymbol{A} = \boldsymbol{e}^T$, multiplying any state vector $\boldsymbol{v}_t$ by $\boldsymbol{A}$ preserves the sum of its entries:

$$\boldsymbol{e}^T\boldsymbol{v}_{t+1} = \boldsymbol{e}^T(\boldsymbol{A}\boldsymbol{v}_t) = (\boldsymbol{e}^T\boldsymbol{A})\boldsymbol{v}_t = \boldsymbol{e}^T\boldsymbol{v}_t.$$

- **Dominant eigenvalue $\lambda_1 = 1$ (Neutral Stability):** Since $\boldsymbol{A} - \boldsymbol{I}$ has columns summing to zero, $\det(\boldsymbol{A} - \boldsymbol{I}) = 0$, so $\lambda_1 = 1$ is always an eigenvalue of $\boldsymbol{A}$. Moreover, all other eigenvalues satisfy $|\lambda_i| \leq 1$ (and if all entries of $\boldsymbol{A}$, or of some power $\boldsymbol{A}^k$, are strictly positive, then $|\lambda_i| < 1$ for all $i \geq 2$ by the Perron–Frobenius theorem).
- **Steady state as the $\lambda = 1$ eigenvector:** As $t \to \infty$, all modes with $|\lambda_i| < 1$ decay to zero ($\lambda_i^t \to 0$), so $\boldsymbol{v}_{t+1} = \boldsymbol{A}\boldsymbol{v}_t$ converges to a **steady-state vector** $\boldsymbol{v}_\infty$ satisfying

$$\boldsymbol{A}\boldsymbol{v}_\infty = 1 \cdot \boldsymbol{v}_\infty = \boldsymbol{v}_\infty.$$

  That is, the steady-state distribution is precisely the eigenvector of $\boldsymbol{A}$ corresponding to $\lambda_1 = 1$ (normalized so its entries sum to the total population or 1).

Example 1.5 (Neutrally Stable Difference Equation: Population Migration Steady State[1])**.** Suppose that every year a fraction p of New Yorkers move to California (so $1 - p$ stay in New York), and a fraction q of Californians move to New York (so $1 - q$ stay in California). The population vector $\boldsymbol{v}_t = (\mathrm{NY}_t, \mathrm{CA}_t)^T$ evolves according to the discrete-time Markov system:

$$\boldsymbol{v}_{t+1} = \boldsymbol{A}\boldsymbol{v}_t, \qquad \begin{pmatrix}\text{New York}_{t+1}\\ \text{California}_{t+1}\end{pmatrix} = \begin{pmatrix}1-p & q\\ p & 1-q\end{pmatrix}\begin{pmatrix}\text{New York}_t\\ \text{California}_t\end{pmatrix}. \tag{5}$$

Notice that each column of $\boldsymbol{A}$ is non-negative and sums to 1: $(1-p)+p = 1$ and $q+(1-q) = 1$.

Now take $p = 0.3$ and $q = 0.1$:

$$\boldsymbol{A} = \begin{pmatrix}1-0.3 & 0.1\\ 0.3 & 1-0.1\end{pmatrix} = \begin{pmatrix}0.7 & 0.1\\ 0.3 & 0.9\end{pmatrix}.$$

[1] Adapted from the interactive visual explanation https://setosa.io/ev/eigenvectors-and-eigenvalues/.

- **Eigenvalues:** Solving the characteristic equation $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$:

$$\begin{aligned}\det(\boldsymbol{A} - \lambda \boldsymbol{I}) &= \begin{vmatrix} 0.7 - \lambda & 0.1 \\ 0.3 & 0.9 - \lambda \end{vmatrix} = (0.7 - \lambda)(0.9 - \lambda) - 0.03 \\ &= \lambda^2 - 1.6\lambda + 0.6 = (\lambda - 1)(\lambda - 0.6) = 0,\end{aligned}$$

  which gives

$$\lambda_1 = 1, \qquad \lambda_2 = 0.6 \quad \text{(in general, } \lambda_2 = 1 - p - q\text{)}.$$

- **Eigenvectors:**
  - For $\lambda_1 = 1$:

$$(\boldsymbol{A} - \boldsymbol{I})\boldsymbol{v}_1 = \begin{pmatrix} -0.3 & 0.1 \\ 0.3 & -0.1 \end{pmatrix} \boldsymbol{v}_1 = \boldsymbol{0} \quad \Longrightarrow \quad \boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 3 \end{pmatrix} \quad \left(\text{or } \boldsymbol{v}_1 = \begin{pmatrix} q \\ p \end{pmatrix} \right).$$

    Normalizing the entries to sum to 1 gives the **steady-state probability vector**:

$$\boldsymbol{v}_\infty = \frac{1}{1+3} \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 0.25 \\ 0.75 \end{pmatrix} \quad \left(\text{in general, } \boldsymbol{v}_\infty = \frac{1}{p+q} \begin{pmatrix} q \\ p \end{pmatrix} \right).$$

  - For $\lambda_2 = 0.6$:

$$(\boldsymbol{A} - 0.6\boldsymbol{I})\boldsymbol{v}_2 = \begin{pmatrix} 0.1 & 0.1 \\ 0.3 & 0.3 \end{pmatrix} \boldsymbol{v}_2 = \boldsymbol{0} \quad \Longrightarrow \quad \boldsymbol{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

- **Convergence to Steady State:** For any initial population distribution $\boldsymbol{v}_0$ whose entries sum to 1 (e.g., $\boldsymbol{v}_0 = (1, 0)^T$), expanding $\boldsymbol{v}_0$ in the eigenvector basis yields

$$\boldsymbol{v}_0 = \begin{pmatrix} 0.25 \\ 0.75 \end{pmatrix} + c_2 \begin{pmatrix} 1 \\ -1 \end{pmatrix} \quad \Longrightarrow \quad \boldsymbol{v}_t = \boldsymbol{A}^t \boldsymbol{v}_0 = 1^t \begin{pmatrix} 0.25 \\ 0.75 \end{pmatrix} + c_2 (0.6)^t \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

  As $t \to \infty$, $(0.6)^t \to 0$, so regardless of the initial population split, the system converges to the steady state where 25% of the population lives in New York and 75% lives in California:

$$\boldsymbol{A}\boldsymbol{v}_\infty = \begin{pmatrix} 0.7 & 0.1 \\ 0.3 & 0.9 \end{pmatrix} \begin{pmatrix} 0.25 \\ 0.75 \end{pmatrix} = \begin{pmatrix} 0.175 + 0.075 \\ 0.075 + 0.675 \end{pmatrix} = \begin{pmatrix} 0.25 \\ 0.75 \end{pmatrix} = \boldsymbol{v}_\infty.$$

  At this equilibrium, the fraction of the total population moving from New York to California each year ($p \cdot 0.25 = 0.3 \times 0.25 = 0.075$) exactly balances the fraction moving from California to New York ($q \cdot 0.75 = 0.1 \times 0.75 = 0.075$).

1.5 Applications to Systems of Differential Equations

Consider a system of first-order linear ordinary differential equations (ODEs):

$$\frac{d\boldsymbol{u}}{dt} = \boldsymbol{A}\boldsymbol{u}(t), \qquad \boldsymbol{u}(0) = \boldsymbol{u}_0.$$

If $\boldsymbol{A}$ has eigenvalues λ_i and eigenvectors $\boldsymbol{x}_i$ (or $\boldsymbol{v}_i$), the general solution is a linear combination of exponential modes:

$$\boldsymbol{u}(t) = c_1 e^{\lambda_1 t} \boldsymbol{x}_1 + c_2 e^{\lambda_2 t} \boldsymbol{x}_2 + \cdots + c_n e^{\lambda_n t} \boldsymbol{x}_n. \tag{6}$$

Remark 1.4 (Stability of ODE Systems vs. Difference Equations). Since $\left|e^{\lambda_i t}\right| = e^{\mathrm{Re}(\lambda_i)t}$, the stability threshold for continuous-time ODEs $\frac{d\boldsymbol{u}}{dt} = \boldsymbol{A}\boldsymbol{u}$ is $\mathrm{Re}(\lambda_i) = 0$ (rather than $|\lambda_i| = 1$ for discrete-time difference equations):

- **Stable:** $\mathrm{Re}(\lambda_i) < 0$ for all i ($\boldsymbol{u}(t) \to \boldsymbol{0}$ as $t \to \infty$).
- **Neutrally stable:** $\mathrm{Re}(\lambda_i) \leq 0$ for all i, with at least one non-defective eigenvalue having $\mathrm{Re}(\lambda_i) = 0$.
- **Unstable:** $\mathrm{Re}(\lambda_i) > 0$ for at least one i ($\|\boldsymbol{u}(t)\| \to \infty$ as $t \to \infty$).

Example 1.6 (Continuous-Time Compartment System[2])**.** Consider the coupled ODE system:

$$\begin{cases} \dfrac{du_1}{dt} = -u_1 + 2u_2, \\ \dfrac{du_2}{dt} = \quad u_1 - 2u_2, \end{cases} \qquad \text{with initial condition } \boldsymbol{u}(0) = \begin{pmatrix} u_1(0) \\ u_2(0) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ at } t = 0.$$

In matrix form, $\frac{d\boldsymbol{u}}{dt} = \boldsymbol{A}\boldsymbol{u}$ with

$$\boldsymbol{A} = \begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix}.$$

- **Eigenvalues:** Solving the characteristic equation $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$:

  $$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = \begin{vmatrix} -1-\lambda & 2 \\ 1 & -2-\lambda \end{vmatrix} = (-1-\lambda)(-2-\lambda) - 2 = \lambda^2 + 3\lambda = \lambda(\lambda+3) = 0,$$

  which gives

  $$\lambda_1 = 0, \qquad \lambda_2 = -3 \quad \text{(neutrally stable: } \lambda_1 = 0, \lambda_2 < 0\text{)}.$$

  (Notice that the columns of $\boldsymbol{A}$ sum to zero, expressing conservation of total mass $u_1(t) + u_2(t) = \text{const}$ and guaranteeing $\lambda_1 = 0$.)

- **Eigenvectors:**
  - For $\lambda_1 = 0$: $\begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix} \boldsymbol{x}_1 = \boldsymbol{0} \implies \boldsymbol{x}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$.
  - For $\lambda_2 = -3$: $\begin{pmatrix} 2 & 2 \\ 1 & 1 \end{pmatrix} \boldsymbol{x}_2 = \boldsymbol{0} \implies \boldsymbol{x}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$.

- **Initial Condition and Steady State:** At $t = 0$,

  $$\boldsymbol{u}(0) = c_1 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + c_2 \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \implies \quad c_1 = \frac{1}{3}, \quad c_2 = \frac{1}{3}.$$

  Therefore, the solution for any $t \geq 0$ is

  $$\boldsymbol{u}(t) = \frac{1}{3} e^{0 \cdot t} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \frac{1}{3} e^{-3t} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

  As $t \to \infty$, $e^{-3t} \to 0$, and the system converges to the steady state $\boldsymbol{u}(\infty) = \begin{pmatrix} 2/3 \\ 1/3 \end{pmatrix}$.

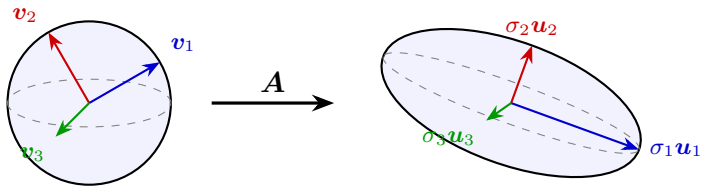
[2] A *compartmental model* (or *compartment system*) divides a system into distinct, well-mixed "compartments" (such as interconnected fluid tanks, organs in pharmacokinetics, or population states in epidemiology) between which mass or individuals flow at specified rates. In this two-compartment example, substance leaves compartment 1 for compartment 2 at rate $1 \cdot u_1$ and flows from compartment 2 back to compartment 1 at rate $2 \cdot u_2$; because the system is closed, every outflow from one compartment is an inflow to the other, so the columns of $\boldsymbol{A}$ sum to zero and the total amount $u_1(t) + u_2(t)$ is conserved.

2 Singular Value Decomposition (SVD)

2.1 Geometric Motivation and Derivation

Whereas eigenvalue decomposition $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{-1}$ (or $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$ for symmetric matrices) applies only to square matrices, the *Singular Value Decomposition (SVD)* generalizes orthogonal diagonalization to any rectangular matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$.

Geometric Motivation: Consider the action of $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ on the unit sphere $\{\boldsymbol{x} \in \mathbb{R}^n : \|\boldsymbol{x}\|_2 = 1\}$. The image $\{\boldsymbol{A}\boldsymbol{x} : \|\boldsymbol{x}\|_2 = 1\}$ is a hyperellipsoid in $\mathbb{R}^m$:



Unit sphere $\|\boldsymbol{x}\|_2 = 1$ Ellipsoid $\{\boldsymbol{A}\boldsymbol{x} : \|\boldsymbol{x}\|_2 = 1\}$

The orthogonal semi-axes of this ellipsoid have directions given by unit vectors $\boldsymbol{u}_i \in \mathbb{R}^m$ and lengths $\sigma_i \geq 0$, which are the images of orthonormal vectors $\boldsymbol{v}_i \in \mathbb{R}^n$ on the unit sphere:

$$\boldsymbol{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i.$$

In matrix form, $\boldsymbol{A}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma}$, and since $\boldsymbol{V}$ has orthonormal columns ($\boldsymbol{V}^T\boldsymbol{V} = \boldsymbol{I}$), multiplying by $\boldsymbol{V}^T$ on the right gives

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T. \tag{7}$$

Algebraic Derivation: Let $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ with $\operatorname{rank}(\boldsymbol{A}) = r$. The matrix $\boldsymbol{A}^T\boldsymbol{A} \in \mathbb{R}^{n\times n}$ is symmetric positive semi-definite, so it has real non-negative eigenvalues and orthonormal eigenvectors:

$$\boldsymbol{A}^T\boldsymbol{A}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i, \quad \|\boldsymbol{v}_i\|_2 = 1, \quad i = 1, \ldots, n,$$

ordered such that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_r > 0$ and $\lambda_{r+1} = \cdots = \lambda_n = 0$.

1. For $i = 1, \ldots, r$, define the *singular values* $\sigma_i = \sqrt{\lambda_i} > 0$.

2. Multiplying $\boldsymbol{A}^T\boldsymbol{A}\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i$ on the left by $\boldsymbol{A}$ shows that $\boldsymbol{w}_i = \boldsymbol{A}\boldsymbol{v}_i$ is an eigenvector of $\boldsymbol{A}\boldsymbol{A}^T$ with eigenvalue λ_i:
$$(\boldsymbol{A}\boldsymbol{A}^T)(\boldsymbol{A}\boldsymbol{v}_i) = \lambda_i(\boldsymbol{A}\boldsymbol{v}_i).$$

3. The vectors $\{\boldsymbol{w}_i\}_{i=1}^r$ are mutually orthogonal and have length $\|\boldsymbol{w}_i\|_2 = \sqrt{\boldsymbol{v}_i^T\boldsymbol{A}^T\boldsymbol{A}\boldsymbol{v}_i} = \sqrt{\lambda_i} = \sigma_i$. Normalizing them defines the *left singular vectors*
$$\boldsymbol{u}_i = \frac{\boldsymbol{A}\boldsymbol{v}_i}{\sigma_i}, \quad i = 1, \ldots, r \quad \Longleftrightarrow \quad \boldsymbol{A}\boldsymbol{v}_i = \sigma_i\boldsymbol{u}_i.$$

4. Completing $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$ to an orthonormal basis $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m\}$ of $\mathbb{R}^m$ (where $\boldsymbol{u}_{r+1}, \ldots, \boldsymbol{u}_m$ span $\mathcal{N}(\boldsymbol{A}\boldsymbol{A}^T) = \mathcal{N}(\boldsymbol{A}^T)$) yields orthogonal matrices $\boldsymbol{U} \in \mathbb{R}^{m\times m}$ and $\boldsymbol{V} \in \mathbb{R}^{n\times n}$ such that
$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_r & \boldsymbol{O} \\ \boldsymbol{O} & \boldsymbol{O} \end{pmatrix} \in \mathbb{R}^{m\times n}, \quad \boldsymbol{\Sigma}_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r).$$

2.2 Key Properties and Eckart–Young–Mirsky Theorem

From $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$, we immediately obtain:

- **Relation to $\boldsymbol{A}^T\boldsymbol{A}$ and $\boldsymbol{A}\boldsymbol{A}^T$:**

$$\boldsymbol{A}^T\boldsymbol{A} = \boldsymbol{V}\boldsymbol{\Sigma}^T\boldsymbol{U}^T\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T = \boldsymbol{V}(\boldsymbol{\Sigma}^T\boldsymbol{\Sigma})\boldsymbol{V}^T, \qquad \boldsymbol{A}\boldsymbol{A}^T = \boldsymbol{U}(\boldsymbol{\Sigma}\boldsymbol{\Sigma}^T)\boldsymbol{U}^T.$$

  Hence $\sigma_i = \sqrt{\lambda_i(\boldsymbol{A}^T\boldsymbol{A})} = \sqrt{\lambda_i(\boldsymbol{A}\boldsymbol{A}^T)}$ for $i = 1, \ldots, r$.

- **Symmetric Matrices:** If $\boldsymbol{A} = \boldsymbol{A}^T$, then $\sigma_i = |\lambda_i(\boldsymbol{A})|$.

- **Rank-1 Outer Product Expansion:**

$$\boldsymbol{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

- **Spectral Norm and Best Low-Rank Approximation (Eckart–Young–Mirsky Theorem):** The 2-norm of $\boldsymbol{A}$ is $\|\boldsymbol{A}\|_2 = \sigma_1$. For any $k \leq r$, the truncated SVD

$$\boldsymbol{A}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$$

  is the best rank-k approximation to $\boldsymbol{A}$ in the 2-norm, with error equal to the $(k+1)$-st singular value:

$$\|\boldsymbol{A} - \boldsymbol{A}_k\|_2 = \left\| \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T - \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T \right\|_2 = \left\| \sum_{i=k+1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T \right\|_2 = \sigma_{k+1}.$$

2.3 Computing the SVD via Golub–Kahan Bidiagonalization

While $\sigma_i = \sqrt{\lambda_i(\boldsymbol{A}^T\boldsymbol{A})}$ defines the singular values theoretically, **explicitly forming $\boldsymbol{A}^T\boldsymbol{A}$ is numerically unstable** because it squares the condition number: $\kappa_2(\boldsymbol{A}^T\boldsymbol{A}) = \kappa_2(\boldsymbol{A})^2$, causing severe loss of accuracy for small singular values in floating-point arithmetic.

Instead, practical SVD algorithms (such as the Golub–Kahan–Reinsch algorithm) proceed in two phases without ever forming $\boldsymbol{A}^T\boldsymbol{A}$:

1. **Phase 1: Reduction to Upper Bidiagonal Form.** Using alternating left and right Householder reflections $\boldsymbol{P}_1, \boldsymbol{P}_2, \ldots$ and $\boldsymbol{Q}_1, \boldsymbol{Q}_2, \ldots$, the matrix $\boldsymbol{A}$ is reduced to an *upper bidiagonal* matrix $\boldsymbol{B}$:

$$\boldsymbol{A} = \boldsymbol{P}\boldsymbol{B}\boldsymbol{Q}^T, \qquad \text{where } \boldsymbol{P}, \boldsymbol{Q} \text{ are orthogonal and } \boldsymbol{B} = \begin{pmatrix} \times & \times & 0 & 0 \\ 0 & \times & \times & 0 \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{pmatrix}.$$

   Specifically, for a 4×4 matrix:

   - Multiply on the left by a Householder reflector $\boldsymbol{P}_1$ to zero out entries below the diagonal in column 1:

$$\boldsymbol{A} = \begin{pmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{pmatrix} \xrightarrow{\boldsymbol{P}_1^T\boldsymbol{A}} \begin{pmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{pmatrix}.$$

- Multiply on the right by a Householder reflector $\boldsymbol{Q}_1$ acting on columns $2, \ldots, n$ to zero out entries to the right of the superdiagonal in row 1 (without destroying the zeros in column 1!):

$$\xrightarrow{\boldsymbol{P}_1^T \boldsymbol{A} \boldsymbol{Q}_1} \begin{pmatrix} \times & \times & 0 & 0 \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{pmatrix}.$$

- Repeat for the remaining columns and rows to obtain $\boldsymbol{B} = \boldsymbol{P}^T \boldsymbol{A} \boldsymbol{Q}$.

2. **Phase 2: SVD of the Bidiagonal Matrix.** Note that $\boldsymbol{A}^T \boldsymbol{A} = \boldsymbol{Q} \boldsymbol{B}^T \boldsymbol{P}^T \boldsymbol{P} \boldsymbol{B} \boldsymbol{Q}^T = \boldsymbol{Q} (\boldsymbol{B}^T \boldsymbol{B}) \boldsymbol{Q}^T$, so $\boldsymbol{B}$ has the exact same singular values as $\boldsymbol{A}$. Dedicated iterative algorithms (implicit-shift QR / Golub–Kahan SVD step, or divide-and-conquer) compute the SVD of the bidiagonal matrix $\boldsymbol{B}$:

$$\boldsymbol{B} = \boldsymbol{X} \boldsymbol{\Sigma} \boldsymbol{Y}^T \quad \text{(computed implicitly without forming } \boldsymbol{B}^T \boldsymbol{B}\text{)}.$$

Substituting $\boldsymbol{B} = \boldsymbol{X} \boldsymbol{\Sigma} \boldsymbol{Y}^T$ back into $\boldsymbol{A} = \boldsymbol{P} \boldsymbol{B} \boldsymbol{Q}^T$ yields the full SVD of $\boldsymbol{A}$:

$$\boldsymbol{A} = \boldsymbol{P} (\boldsymbol{X} \boldsymbol{\Sigma} \boldsymbol{Y}^T) \boldsymbol{Q}^T = \underbrace{(\boldsymbol{P} \boldsymbol{X})}_{\boldsymbol{U}} \boldsymbol{\Sigma} \underbrace{(\boldsymbol{Q} \boldsymbol{Y})^T}_{\boldsymbol{V}^T}.$$

3 Principal Component Analysis (PCA)

3.1 Data Matrix and Basic Statistics

Suppose we have a dataset consisting of m *samples* (observations, rows) and n *features* (variables/attributes, columns), stored in a raw data matrix $\widetilde{\boldsymbol{X}} \in \mathbb{R}^{m \times n}$:

$$\widetilde{\boldsymbol{X}} = \begin{pmatrix} — & \tilde{\boldsymbol{x}}_1^T & — \\ — & \tilde{\boldsymbol{x}}_2^T & — \\ & \vdots & \\ — & \tilde{\boldsymbol{x}}_m^T & — \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

To review basic sample statistics, suppose we have two features (columns of $\widetilde{\boldsymbol{X}}$), denoted $\boldsymbol{a} = (a_1, a_2, \ldots, a_m)^T$ (e.g., height) and $\boldsymbol{b} = (b_1, b_2, \ldots, b_m)^T$ (e.g., weight), so each sample is a row vector $\tilde{\boldsymbol{x}}_i^T = (a_i, b_i)$:

1. **Sample Mean (Estimate of Expectation):**

$$\mu_A = \frac{1}{m} \sum_{i=1}^{m} a_i.$$

2. **Sample Variance:** Measures the spread of feature A around its mean:

$$\mathrm{Var}(A) = \frac{1}{m-1} \sum_{i=1}^{m} (a_i - \mu_A)^2 = \frac{1}{m-1} \left((a_1 - \mu_A)^2 + \cdots + (a_m - \mu_A)^2 \right),$$

where dividing by $m-1$ instead of m is *Bessel's correction*, which provides an unbiased estimator of the population variance when the mean μ_A is estimated from the same sample.

3. **Sample Covariance:** Measures the joint linear variability of two features A and B:

$$\mathrm{Cov}(A, B) = \frac{1}{m-1} \sum_{i=1}^{m} (a_i - \mu_A)(b_i - \mu_B).$$

Note that $\mathrm{Cov}(A, A) = \mathrm{Var}(A)$ and $\mathrm{Cov}(A, B) = \mathrm{Cov}(B, A)$.

3.2 Centering the Data and the Covariance Matrix

Let $\boldsymbol{\mu} \in \mathbb{R}^{1\times n}$ be the row vector of column means:

$$\boldsymbol{\mu} = \left(\frac{1}{m}\sum_{i=1}^{m} \tilde{x}_{i,1},\ \frac{1}{m}\sum_{i=1}^{m} \tilde{x}_{i,2},\ \ldots,\ \frac{1}{m}\sum_{i=1}^{m} \tilde{x}_{i,n} \right).$$

We **center** the dataset by subtracting the mean row $\boldsymbol{\mu}$ from every row of $\widetilde{\boldsymbol{X}}$:

$$\boldsymbol{X} = \widetilde{\boldsymbol{X}} - \begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu} \\ \vdots \\ \boldsymbol{\mu} \end{pmatrix} \in \mathbb{R}^{m\times n},$$

so that each column of $\boldsymbol{X}$ has zero mean.

The **sample covariance matrix** $\boldsymbol{C} \in \mathbb{R}^{n\times n}$ of the centered data matrix $\boldsymbol{X}$ is then given concisely by

$$\boldsymbol{C} = \frac{1}{m-1}\boldsymbol{X}^T\boldsymbol{X} = \begin{pmatrix} \mathrm{Cov}(A,A) & \mathrm{Cov}(A,B) & \cdots \\ \mathrm{Cov}(B,A) & \mathrm{Cov}(B,B) & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}. \tag{8}$$

Here the diagonal entries $C_{jj} = \mathrm{Var}(\text{feature } j)$ are the variances of the individual features, and the off-diagonal entries C_{jk} are the covariances between pairs of features.

3.3 PCA via Eigen-Decomposition and SVD

Since $\boldsymbol{C} = \frac{1}{m-1}\boldsymbol{X}^T\boldsymbol{X}$ is symmetric positive semi-definite, it admits an orthogonal eigenvalue decomposition:

$$\boldsymbol{C} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T, \qquad \boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n), \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0. \tag{9}$$

Now consider the Singular Value Decomposition of the centered data matrix $\boldsymbol{X}$:

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T.$$

Substituting $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ into the covariance matrix (8) gives

$$\boldsymbol{C} = \frac{1}{m-1}\boldsymbol{X}^T\boldsymbol{X} = \frac{1}{m-1}\boldsymbol{V}\boldsymbol{\Sigma}^T\underbrace{\boldsymbol{U}^T\boldsymbol{U}}_{\boldsymbol{I}}\boldsymbol{\Sigma}\boldsymbol{V}^T = \boldsymbol{V}\left(\frac{\boldsymbol{\Sigma}^2}{m-1}\right)\boldsymbol{V}^T. \tag{10}$$

Comparing $\boldsymbol{C} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T$ with $\boldsymbol{C} = \boldsymbol{V}\frac{\boldsymbol{\Sigma}^2}{m-1}\boldsymbol{V}^T$ reveals the direct connection between PCA and SVD:

1. **Principal Axes (Directions):** The right singular vectors of $\boldsymbol{X}$ (columns of $\boldsymbol{V} = (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n)$) are the eigenvectors of the covariance matrix $\boldsymbol{C}$. They define the new orthonormal basis of uncorrelated directions maximizing variance.

2. **Eigenvalues (Variances along Principal Axes):** The eigenvalues of $\boldsymbol{C}$ are related to the singular values of $\boldsymbol{X}$ by

$$\lambda_i = \frac{\sigma_i^2}{m-1}.$$

3. **Principal Components (Projected Data / Scores):** Multiplying $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ on the right by $\boldsymbol{V}$ gives

$$\boxed{\boldsymbol{X}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma} = (\sigma_1\boldsymbol{u}_1,\ \sigma_2\boldsymbol{u}_2,\ \ldots,\ \sigma_n\boldsymbol{u}_n).}$$

The matrix $\boldsymbol{X}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma} \in \mathbb{R}^{m\times n}$ represents the **change of basis** of the rows of $\boldsymbol{X}$ into the orthonormal basis formed by the columns of $\boldsymbol{V}$:

- Each **row** j of $\boldsymbol{X}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma}$ is the j-th data sample $\boldsymbol{x}_j^T\boldsymbol{V}$ expressed in the new coordinates (the principal axes).
- Each **column** i of $\boldsymbol{X}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma}$ is $\boldsymbol{X}\boldsymbol{v}_i = \sigma_i\boldsymbol{u}_i \in \mathbb{R}^m$ (the i-th column of $\boldsymbol{U}$ *scaled by* σ_i), which gives the projections (scores) of all m samples onto the i-th principal axis $\boldsymbol{v}_i$; these columns $\sigma_i\boldsymbol{u}_i$ are called the *principal components*.

To reduce dimensionality from n features to $k < n$ features, we keep only the first k columns:

$$\boldsymbol{X}_k = \boldsymbol{X}\boldsymbol{V}_k = \boldsymbol{U}_k\boldsymbol{\Sigma}_k = (\sigma_1\boldsymbol{u}_1, \ldots, \sigma_k\boldsymbol{u}_k) \in \mathbb{R}^{m\times k}.$$

3.4 Total Variance and Explained Variance Ratio

Since the trace of a matrix is invariant under orthogonal similarity transformations ($\mathrm{tr}(\boldsymbol{C}) = \mathrm{tr}(\boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T) = \mathrm{tr}(\boldsymbol{\Lambda})$), the **total variance** T of the dataset equals both the sum of the original feature variances and the sum of the eigenvalues of $\boldsymbol{C}$:

$$T = \mathrm{tr}(\boldsymbol{C}) = \sum_{i=1}^{n} \mathrm{Var}(\text{feature } i) = \sum_{i=1}^{n} \lambda_i = \frac{1}{m-1}\sum_{i=1}^{n} \sigma_i^2. \tag{11}$$

The fraction of the total variance captured by the i-th principal component $\boldsymbol{v}_i$ is the **explained variance ratio**:

$$\frac{\lambda_i}{T} = \frac{\lambda_i}{\sum_{j=1}^{n}\lambda_j} = \frac{\sigma_i^2}{\sum_{j=1}^{n}\sigma_j^2}.$$

Example 3.1 (2D Covariance Matrices and Principal Axes)**.** Consider two 2D datasets with total variance $T = 100$:

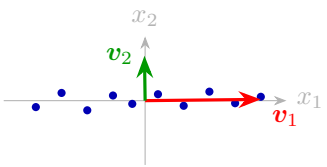
1. **Nearly uncorrelated features aligned with coordinate axes:**

$$\boldsymbol{C}_1 = \begin{pmatrix} 95 & 1 \\ 1 & 5 \end{pmatrix}.$$

Here the data spread is almost entirely along the horizontal axis:

$$\lambda_1 \approx 95, \quad \lambda_2 \approx 5, \qquad \boldsymbol{v}_1 \approx \begin{pmatrix} 0.999 \\ 0.011 \end{pmatrix}, \quad \boldsymbol{v}_2 \approx \begin{pmatrix} -0.011 \\ 0.999 \end{pmatrix}.$$

The first principal component $\boldsymbol{v}_1$ captures $\lambda_1/T \approx 95\%$ of the total variance.



2. **Strongly correlated features tilted at 45°:**

$$\boldsymbol{C}_2 = \begin{pmatrix} 50 & 40 \\ 40 & 50 \end{pmatrix}.$$

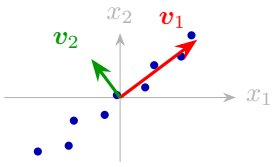
Each individual coordinate has variance 50, but they covary strongly (Cov = 40). Computing the eigenvalues via $\det(\boldsymbol{C}_2 - \lambda \boldsymbol{I}) = 0$:

$$\det(\boldsymbol{C}_2 - \lambda \boldsymbol{I}) = \begin{vmatrix} 50-\lambda & 40 \\ 40 & 50-\lambda \end{vmatrix} = (50-\lambda)^2 - 40^2 = 0 \quad \Longrightarrow \quad 50 - \lambda = \pm 40,$$

which gives:

$$\lambda_1 = 90, \quad \boldsymbol{v}_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}; \qquad \lambda_2 = 10, \quad \boldsymbol{v}_2 = \begin{pmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}.$$

The total variance is $T = 50 + 50 = \lambda_1 + \lambda_2 = 90 + 10 = 100$. Projecting onto the 45° diagonal $\boldsymbol{v}_1$ captures $\frac{\lambda_1}{T} = \frac{90}{100} = 90\%$ of the total variance in a single coordinate!



Example 3.2 (PCA on the Iris Dataset (`107/pca_svd.py`))**.** Consider the classical Iris dataset consisting of $m = 150$ flower samples across 3 species (*setosa*, *versicolor*, *virginica*) measured over $n = 4$ morphological features (in cm). After centering the 150×4 data matrix $\boldsymbol{X}$, the sample covariance matrix $\boldsymbol{C} = \frac{1}{m-1}\boldsymbol{X}^T\boldsymbol{X} \in \mathbb{R}^{4\times 4}$ has total variance $T = \mathrm{tr}(\boldsymbol{C}) = 4.573$ cm^2. Computing the SVD $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ yields the feature variances $\mathrm{diag}(\boldsymbol{C})$, eigenvalues $\lambda_i = \sigma_i^2/(m-1)$, and principal axes (columns of $\boldsymbol{V}$):

Feature	**Variance**	**1st Axis $\boldsymbol{v}_1$**	**2nd Axis $\boldsymbol{v}_2$**	**3rd Axis $\boldsymbol{v}_3$**
`sepal length (cm)`	0.686	+0.361	−0.657	−0.582
`sepal width (cm)`	0.190	−0.085	−0.730	+0.598
`petal length (cm)`	**3.116**	**+0.857**	+0.173	+0.076
`petal width (cm)`	0.581	+0.358	+0.075	+0.546
Eigenvalue		4.228	0.243	0.078
Explained Variance		**92.46**%	**5.31**%	1.71%

- **First principal axis $\boldsymbol{v}_1$ (92.46% of total variance):** $\boldsymbol{v}_1$ is heavily dominated by `petal length` (+0.857), which by itself accounts for 3.116/4.573 = 68.1% of the dataset's variance. Because `petal length` is strongly positively correlated with `sepal length` (+0.361) and `petal width` (+0.358), moving along $\boldsymbol{v}_1$ increases all three dimensions simultaneously (overall flower/petal size), while `sepal width` (−0.085) slightly decreases.
- **Second principal axis $\boldsymbol{v}_2$ (5.31% of total variance):** Orthogonal to $\boldsymbol{v}_1$, the second principal axis $\boldsymbol{v}_2$ is dominated by `sepal width` (−0.730) and `sepal length` (−0.657), capturing sepal variation that is independent of petal size.
- **Dimensionality reduction to $k = 2$:** Projecting the 4D data onto the first two columns $\boldsymbol{V}_2 = (\boldsymbol{v}_1, \boldsymbol{v}_2)$ gives $\boldsymbol{X}_2 = \boldsymbol{X}\boldsymbol{V}_2 = \boldsymbol{U}_2\boldsymbol{\Sigma}_2 \in \mathbb{R}^{150\times 2}$, which preserves $(4.228+0.243)/4.573 =$ **97.77**% of the total variance while discarding only 2.23% along $\boldsymbol{v}_3$ and $\boldsymbol{v}_4$.

Remark 3.1 (Covariance Matrix vs. Correlation Matrix in PCA). Because PCA seeks directions that maximize numerical variance, it is **not scale-invariant**: multiplying a feature by 10 (e.g., changing units from centimeters to millimeters) multiplies its variance by 100 and causes the first principal axis $\boldsymbol{v}_1$ to align almost entirely with that feature. Consequently, one must choose between two variants of PCA depending on the nature of the data:

1. **PCA on the Covariance Matrix $\boldsymbol{C} = \frac{1}{m-1}\boldsymbol{X}^T\boldsymbol{X}$ (mean-centered data):**
   - **When to use:** All features are measured in the *same physical units* and have comparable scales, so differences in feature variances carry meaningful physical information (as in the Iris dataset above, where all four features are lengths in cm, and the larger variance of `petal length` reflects genuine biological variation).
   - **Total variance:** $T = \mathrm{tr}(\boldsymbol{C}) = \sum_{j=1}^{n} s_j^2$, where $s_j = \sqrt{\mathrm{Var}(\text{feature } j)}$ is the sample standard deviation of the j-th feature.
2. **PCA on the Correlation Matrix $\boldsymbol{R} = \frac{1}{m-1}\boldsymbol{Z}^T\boldsymbol{Z}$ (standardized / z-scored data):**
   - **When to use:** Features are measured in *different or incommensurable units* (e.g., height in m, weight in kg, income in \$, age in years) or their variances differ by orders of magnitude, and we want every feature to contribute equally on a dimensionless scale.
   - **Formulation:** Each centered feature column is divided by its sample standard deviation $s_j = \sqrt{C_{jj}}$ to form the standardized (z-score) data matrix $\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{D}^{-1}$, where $\boldsymbol{D} = \mathrm{diag}(s_1, \ldots, s_n)$ and entry $z_{ij} = (\tilde{x}_{ij} - \mu_j)/s_j$ has mean 0 and unit variance (1). Its covariance matrix is the **sample correlation matrix**:
   $$\boldsymbol{R} = \frac{1}{m-1}\boldsymbol{Z}^T\boldsymbol{Z} = \boldsymbol{D}^{-1}\boldsymbol{C}\boldsymbol{D}^{-1}, \qquad R_{jk} = \frac{\mathrm{Cov}(j,k)}{s_j s_k} \in [-1,1], \quad R_{jj} = 1.$$
   - **Total variance:** Every standardized feature has unit variance ($R_{jj} = 1$), so the total variance is simply the number of features, $T = \mathrm{tr}(\boldsymbol{R}) = n$, and an eigenvalue $\lambda_i > 1$ indicates a principal component that captures more variance than a single original variable.
   - **Important caveat:** The eigenvalues and eigenvectors of $\boldsymbol{R}$ and $\boldsymbol{C}$ are *not* related by a simple rescaling (since $\boldsymbol{D}$ is generally not a scalar multiple of $\boldsymbol{I}$, $\boldsymbol{D}^{-1}$ does not preserve orthogonality), so covariance-PCA and correlation-PCA yield genuinely different principal subspaces.

4 PageRank Algorithm

4.1 Introduction, Network Centrality, and Link Democracy

Introduced by Larry Page and Sergey Brin in 1998 as the foundational ranking algorithm of the Google search engine, **PageRank** addresses a fundamental problem in network analysis: *how to measure the global importance (centrality) of nodes in a directed graph $G = (V, E)$ purely from its link structure.* While originally formulated for the World Wide Web—where vertices $V = \{1, 2, \ldots, N\}$ are web pages and directed edges $y \to x$ are hyperlinks—PageRank applies to **any directed network**:

- **World Wide Web & Information Retrieval:** Ranking web pages by authority so that search results reflect global consensus rather than just keyword frequency.

- **Bibliometrics & Citation Networks:** Evaluating the impact of scientific papers, authors, or journals, where $y \to x$ means paper y cites paper x (e.g., the *Eigenfactor* journal metric).
- **Social & Trust Networks:** Identifying influential users, communities, or trust propagation in social networks (e.g., recommendation of accounts to follow).
- **Bioinformatics & Systems Biology:** Prioritizing candidate disease genes or essential proteins in protein–protein interaction (PPI), metabolic, and gene regulatory networks (*GeneRank*).
- **Recommender Systems & Infrastructure Networks:** Computing personalized rankings on user–item graphs (*Personalized PageRank*), or identifying critical bottlenecks in road, power-grid, and financial networks.

To rank the nodes of $G = (V, E)$, PageRank assigns to every node $x \in \{1, \ldots, N\}$ a non-negative numerical value $PR(x) \geq 0$, called the **PageRank score** (or **importance score**) of node x. These scores are normalized to sum to one across the entire network:

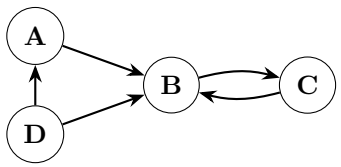
$$PR(x) \geq 0 \quad (\forall x \in V), \qquad \sum_{x=1}^{N} PR(x) = 1,$$

so that $PR(x)$ represents node x's relative share of the total importance in the network (and, as we will see in Section 4.2, the long-run probability of finding a random walker at node x). PageRank defines $PR(x)$ through two core principles:

1. **Link Democracy (Links as Votes):** A directed edge $y \to x$ is treated as a "vote" of confidence or endorsement from node y to node x. We represent the graph topology by the binary **adjacency matrix** $\boldsymbol{A} \in \{0, 1\}^{N \times N}$, where

$$A_{ij} = \begin{cases} 1 & \text{if node } j \text{ links to node } i \ (j \to i), \\ 0 & \text{otherwise.} \end{cases}$$

   Note the convention: **column** j lists the outgoing links from node j, while **row** i lists the incoming links pointing to node i. Thus the out-degree of node j is the j-th column sum, $\text{out}(j) = \sum_{i=1}^{N} A_{ij}$. For example, on the 4-node graph with edges $A \to B$, $B \leftrightarrow C$, $D \to A$, and $D \to B$:



$$\boldsymbol{A} = \begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} A & B & C & D \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

2. **Recursive Importance (Defining $PR(x)$):** Simply counting incoming links (in-degree) treats all votes as equal. In PageRank, the weight of a vote from node y is proportional to node y's own importance score $PR(y)$, divided equally among its $\text{out}(y) > 0$ outgoing links so that each target node x ($y \to x$) receives a share $\frac{PR(y)}{\text{out}(y)}$. The PageRank score $PR(x)$ of node x is defined self-consistently as the sum of the vote shares it receives from all nodes y pointing to x:

$$PR(x) = \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}, \qquad x = 1, \ldots, N. \tag{12}$$

4.2 Transition Matrix, Random Walk $\boldsymbol{p}(t)$, and the Dominant Eigenvector $\boldsymbol{r}$

To write the node-level equations (12) in matrix form—and to connect $PR(x)$, the random-walk distribution $\boldsymbol{p}(t)$, and the stationary PageRank vector $\boldsymbol{r}$—assuming every node has $\text{out}(j) \geq 1$, we normalize each column j of the adjacency matrix $\boldsymbol{A}$ by its out-degree $\text{out}(j)$ to form the **transition (Markov) matrix** $\boldsymbol{M} \in \mathbb{R}^{N\times N}$:

$$M_{ij} = \frac{A_{ij}}{\text{out}(j)} = \begin{cases} \dfrac{1}{\text{out}(j)} & \text{if } j \to i, \\ 0 & \text{otherwise,} \end{cases} \qquad \Longleftrightarrow \qquad \boldsymbol{M} = \boldsymbol{A}\boldsymbol{D}_{\text{out}}^{-1}, \tag{13}$$

where $\boldsymbol{D}_{\text{out}} = \text{diag}(\text{out}(1), \ldots, \text{out}(N))$. Since $M_{ij} \geq 0$ and each column j sums to $\sum_{i=1}^{N} M_{ij} = 1$, the matrix $\boldsymbol{M}$ is a **column-stochastic (Markov) matrix** in the sense of Section 1.4.

Link Between the Random Walk $\boldsymbol{p}(t)$, the Vector $\boldsymbol{r}$, and $PR(x)$. Consider a **random surfer** (or random walker on the network) who starts at step $t = 0$ with an initial probability distribution $\boldsymbol{p}(0) \in \mathbb{R}^N$ and, at each discrete step $t \to t+1$, moves from their current node y to one of its outgoing neighbors x ($y \to x$) chosen uniformly at random with probability $\frac{1}{\text{out}(y)}$:

1. **Time-dependent state vector $\boldsymbol{p}(t)$:** Let $p_x(t) = \mathbb{P}(\text{walker is at node } x \text{ at step } t)$. Collecting these probabilities for all N nodes gives the state distribution vector at time t:

$$\boldsymbol{p}(t) = \begin{pmatrix} p_1(t) \\ p_2(t) \\ \vdots \\ p_N(t) \end{pmatrix} \in \mathbb{R}^N, \qquad p_x(t) \geq 0, \quad \sum_{x=1}^{N} p_x(t) = \boldsymbol{e}^T \boldsymbol{p}(t) = 1.$$

   By the law of total probability, the probability of arriving at node x at step $t+1$ is

$$p_x(t+1) = \sum_{y\to x} \frac{p_y(t)}{\text{out}(y)} = \sum_{y=1}^{N} M_{xy}\, p_y(t) \qquad \Longleftrightarrow \qquad \boxed{\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t).}$$

2. **Stationary PageRank vector $\boldsymbol{r}$ and scalar scores $PR(x)$:** As $t \to \infty$, we seek a *steady-state (stationary) probability distribution* $\boldsymbol{r} = \lim_{t\to\infty} \boldsymbol{p}(t)$ that remains invariant under one step of the Markov chain ($\boldsymbol{p}(t+1) = \boldsymbol{p}(t) = \boldsymbol{r}$). The **PageRank $PR(x)$ of node x is simply the x-th entry r_x of this stationary probability vector $\boldsymbol{r}$**:

$$\boxed{\boldsymbol{r} = \lim_{t\to\infty} \boldsymbol{p}(t) = \begin{pmatrix} r_1 \\ r_2 \\ \vdots \\ r_N \end{pmatrix} = \begin{pmatrix} PR(1) \\ PR(2) \\ \vdots \\ PR(N) \end{pmatrix}, \qquad r_x = PR(x) \geq 0, \quad \sum_{x=1}^{N} PR(x) = \boldsymbol{e}^T \boldsymbol{r} = 1.} \tag{14}$$

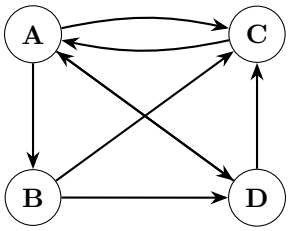
   In iterative computations (Power Iteration), the k-th iterate $\boldsymbol{p}(k)$ is written componentwise as $PR_k(x) := p_x(k)$, so that $\boldsymbol{p}(k) = (PR_k(1), \ldots, PR_k(N))^T \to \boldsymbol{r}$ as $k \to \infty$.

3. **PageRank as the Dominant Eigenvector Problem:** Substituting $r_x = PR(x)$ into the node-level balance equations (12) (or setting $\boldsymbol{p}(t+1) = \boldsymbol{p}(t) = \boldsymbol{r}$ in $\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t)$) reveals that computing PageRank is equivalent to finding the **dominant right eigenvector of $\boldsymbol{M}$ associated with the dominant eigenvalue $\mu_1 = 1$**:

$$r_x = \sum_{y=1}^{N} M_{xy} r_y \quad (\forall x) \qquad \Longleftrightarrow \qquad \boxed{\boldsymbol{M}\boldsymbol{r} = 1 \cdot \boldsymbol{r}, \qquad \|\boldsymbol{r}\|_1 = \boldsymbol{e}^T \boldsymbol{r} = 1.} \tag{15}$$

Recall from Section 1.4 that every column-stochastic matrix has spectral radius $\rho(\boldsymbol{M}) = 1$, with $\mu_1 = 1$ guaranteed to be an eigenvalue because $\boldsymbol{e}^T\boldsymbol{M} = \boldsymbol{e}^T$.

Example 4.1 (4-Node Graph: From Recursive Equations to the Dominant Eigenvector of $\boldsymbol{M}$)**.** Consider a 4-node directed graph with vertices A, B, C, D:



The out-degrees are $\text{out}(A) = 3$ ($A \to B, C, D$), $\text{out}(B) = 2$ ($B \to C, D$), $\text{out}(C) = 1$ ($C \to A$), and $\text{out}(D) = 2$ ($D \to A, C$). The recursive PageRank equations (12) are:

$$PR(A) = PR(C) + \frac{PR(D)}{2}, \qquad PR(B) = \frac{PR(A)}{3},$$
$$PR(C) = \frac{PR(A)}{3} + \frac{PR(B)}{2} + \frac{PR(D)}{2}, \qquad PR(D) = \frac{PR(A)}{3} + \frac{PR(B)}{2}.$$

Collecting the unknowns into $\boldsymbol{r} = (PR(A), PR(B), PR(C), PR(D))^T$, this system is equivalent to the dominant eigenvector problem $\boldsymbol{M}\boldsymbol{r} = 1 \cdot \boldsymbol{r}$:

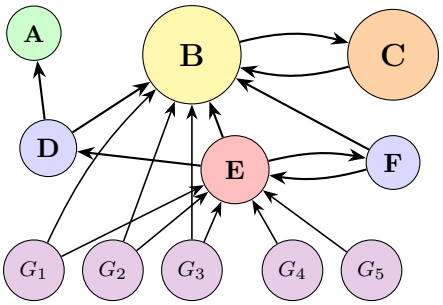
$$\underbrace{\begin{pmatrix} 0 & 0 & 1 & 1/2 \\ 1/3 & 0 & 0 & 0 \\ 1/3 & 1/2 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{pmatrix}}_{\boldsymbol{M}} \underbrace{\begin{pmatrix} PR(A) \\ PR(B) \\ PR(C) \\ PR(D) \end{pmatrix}}_{\boldsymbol{r}} = 1 \cdot \underbrace{\begin{pmatrix} PR(A) \\ PR(B) \\ PR(C) \\ PR(D) \end{pmatrix}}_{\boldsymbol{r}}.$$

Solving $(\boldsymbol{M} - \boldsymbol{I})\boldsymbol{r} = \boldsymbol{0}$ subject to $PR(A) + PR(B) + PR(C) + PR(D) = 1$ gives the unique dominant eigenvector $\boldsymbol{r} = \left(\frac{12}{31}, \frac{4}{31}, \frac{9}{31}, \frac{6}{31}\right)^T \approx (0.387, 0.129, 0.290, 0.194)^T$.

4.3 Pathologies of Pure Link-Following and Teleportation (Damping Factor)

On general directed graphs, pure link-following $\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t)$ fails to produce a unique, meaningful dominant eigenvector $\boldsymbol{r}$ because of two structural pathologies:

Example 4.2 (Random Surfer Walk and Pathologies on the Wikimedia Example Graph[3])**.** Consider the 11-node network consisting of nodes A, B, C, D, E, F and five peripheral nodes $G_1, \ldots, G_5$ that have no incoming links:



[3]Graph from https://commons.wikimedia.org/wiki/File:PageRanks-Example.svg (see also `108/pagerank.py`).

Follow a random surfer starting at one of the bottom nodes G:

$$G \to E \to F \to E \to D \to B \to C \quad \Longrightarrow \quad B \leftrightarrow C \text{ forever!}$$

This graph exhibits both fundamental pathologies of the raw transition matrix $\boldsymbol{M}$:

1. **Spider traps (absorbing cycles / reducible graphs):** Once the surfer enters B or C, the closed cycle $B \leftrightarrow C$ traps them forever (and periodic 2-cycles cause $\boldsymbol{M}$ to have eigenvalue $\mu = -1$ in addition to $\mu = 1$, preventing power iteration from converging!). Furthermore, if a graph has multiple disconnected components or traps, $\mu_1 = 1$ has geometric multiplicity > 1, so the stationary eigenvector is non-unique.
2. **Sink / dangling nodes ($\mathbf{out}(z) = 0$):** Node A has no outgoing links ($\text{out}(A) = 0$), which produces a column of all zeros in $\boldsymbol{M}$. Consequently, $\boldsymbol{M}$ is no longer column-stochastic, its spectral radius drops below 1 (or probability mass leaks out of the graph at A), and $\boldsymbol{p}(t) \to \mathbf{0}$.

Teleportation (Damping Factor λ). To eliminate spider traps and disconnected components, Brin and Page introduced a **damping factor** $\lambda \in (0, 1)$ (typically $\lambda = 0.85$):

- With probability λ, the walker follows an outgoing link uniformly at random ($\boldsymbol{M}\boldsymbol{p}(t)$).
- With probability $1 - \lambda$, the walker **teleports** (jumps) to a node chosen uniformly at random among all N nodes in the network ($\frac{1-\lambda}{N}\boldsymbol{e}$).

At the node level, the time evolution $p_x(t+1)$ and stationary balance $r_x = PR(x)$ become:

$$p_x(t+1) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{p_y(t)}{\text{out}(y)} \quad \xrightarrow{t \to \infty} \quad PR(x) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}. \tag{16}$$

4.4 Power Iteration Algorithm and Handling Sink Nodes

Equation (16) is solved iteratively via the **Power Method**, generating the sequence of probability vectors $\boldsymbol{p}(k) = (PR_k(1), \ldots, PR_k(N))^T$:

1. **Initialize ($k = 0$):** Set $\boldsymbol{p}(0) = \frac{1}{N}\boldsymbol{e}$, i.e., $PR_0(x) = \frac{1}{N}$ for all $x = 1, \ldots, N$.
2. **Iterate ($k = 0, 1, 2, \ldots$) until $\|\boldsymbol{p}(k+1) - \boldsymbol{p}(k)\|_1 < \varepsilon$:**

$$\forall x \in \{1, \ldots, N\}: \quad PR_{k+1}(x) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{PR_k(y)}{\text{out}(y)}.$$

Example 4.3 (First Iterations on the 11-Node Graph: Rapid Convergence)**.** Consider the 11-node Wikimedia graph from Example 4.2 with $N = 11$, $\lambda = 0.85$ ($1 - \lambda = 0.15$), and uniform initial distribution $\boldsymbol{p}(0) = \frac{1}{11}\boldsymbol{e}$ ($PR_0(x) = \frac{1}{11} \approx 0.0909$ for all nodes):

- **Iteration 1 for node B:** Node B receives links from C ($\text{out}(C) = 1$), D ($\text{out}(D) = 2$), E ($\text{out}(E) = 3$), F ($\text{out}(F) = 2$), and G_1, G_2, G_3 ($\text{out}(G_i) = 2$ each):

$$\begin{aligned} PR_1(B) &= 0.15 \cdot \frac{1}{11} + 0.85 \left(PR_0(C) + \frac{PR_0(D)}{2} + \frac{PR_0(E)}{3} + \frac{PR_0(F)}{2} + 3 \cdot \frac{PR_0(G_1)}{2} \right) \\ &= \frac{0.15}{11} + 0.85 \cdot \frac{1}{11} \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{2} + \frac{3}{2} \right) \approx 0.31. \end{aligned}$$

  In a single step, B's rank jumps from $\frac{1}{11} \approx 0.09$ to 0.31 (already close to its exact stationary value $r_B = 0.384$).

- **Iteration 1 for node C:** Node C only receives a link from B ($\text{out}(B) = 1$), which at $k = 0$ still has $PR_0(B) = \frac{1}{11}$:

$$PR_1(C) = \frac{0.15}{11} + 0.85\left(\frac{1}{11}\right) = \frac{1}{11} \approx 0.0909.$$

- **Iteration 2 for node C:** In the second step, B's high rank $PR_1(B) \approx 0.31$ propagates directly to C:

$$PR_2(C) = \frac{0.15}{11} + 0.85 \cdot PR_1(B) \approx \frac{0.15}{11} + 0.85 \times 0.31 \approx 0.28.$$

  Thus, after just **two iterations**, $PR_2(C) \approx 0.28$ has already surged near its final converged value ($r_C = 0.343$), illustrating the extremely fast initial convergence of PageRank power iteration (in `l08/pagerank.py`, $\|\boldsymbol{p}(k+1) - \boldsymbol{p}(k)\|_1$ drops below 0.1 in just 10 iterations).

Problem and Solution for Sink (Dangling) Nodes. Suppose we have a 2-node graph $A \to B$ where B is a sink node ($\text{out}(B) = 0$) and $\lambda = 0.8$. Starting with $PR_0(A) = PR_0(B) = 0.5$:

- For node A (no incoming links): $PR_1(A) = \frac{1-\lambda}{2} + \lambda \cdot 0 = \frac{0.2}{2} = 0.1$.
- For node B (link from A with $\text{out}(A) = 1$):
  - Synchronous update (using $PR_0(A) = 0.5$): $PR_1(B) = 0.1 + 0.8 \times 0.5 = 0.5$, and $PR_2(B) = 0.1 + 0.8 \times 0.1 = 0.18$.
  - Gauss–Seidel update (using $PR_1(A) = 0.1$ immediately): $PR_1(B) = 0.1 + 0.8 \times 0.1 = 0.18$.

In either case, at steady state $PR(A) = 0.1$ and $PR(B) = 0.18$, so $PR(A) + PR(B) = 0.28 \neq 1$! Probability mass leaks out of the system through the zero column of sink node B.

Solution — Stochastic Fix for Sink Nodes: When the walker reaches any sink node z ($\text{out}(z) = 0$, denoted $z \to \emptyset$), they jump uniformly at random to any node in the graph with probability $1/N$. In matrix terms, this replaces every zero column z of $\boldsymbol{M}$ with the uniform probability vector $\frac{1}{N}\boldsymbol{e}$, yielding a patched, strictly column-stochastic matrix $\widetilde{\boldsymbol{M}}$:

$$\widetilde{\boldsymbol{M}} = \boldsymbol{M} + \frac{1}{N}\boldsymbol{e}\,\boldsymbol{d}^T, \qquad \text{where } d_z = \begin{cases} 1 & \text{if } \text{out}(z) = 0, \\ 0 & \text{otherwise.} \end{cases} \tag{17}$$

Crucially, we never form the dense rank-1 correction $\frac{1}{N}\boldsymbol{e}\,\boldsymbol{d}^T$ explicitly; instead, at each iteration we accumulate the total probability mass of all sink nodes into a single scalar $S = \boldsymbol{d}^T\boldsymbol{p}(k) = \sum_{z\to\emptyset} PR_k(z)$:

$$PR_{k+1}(x) = \frac{1-\lambda}{N} + \lambda \sum_{y\to x} \frac{PR_k(y)}{\text{out}(y)} + \lambda \sum_{z\to\emptyset} \frac{PR_k(z)}{N} = \frac{1-\lambda+\lambda S}{N} + \lambda \sum_{y\to x} \frac{PR_k(y)}{\text{out}(y)}. \tag{18}$$

4.5 The Google Matrix, Dominant Eigenvector Theory, and Personalized PageRank

With sink columns replaced by $\frac{1}{N}\boldsymbol{e}$ so that $\widetilde{\boldsymbol{M}}$ is column-stochastic ($\boldsymbol{e}^T\widetilde{\boldsymbol{M}} = \boldsymbol{e}^T$), the damped random-walk evolution (18) and its steady-state limit $\boldsymbol{p}(t) \to \boldsymbol{r}$ take the vector form:

$$\boldsymbol{p}(t+1) = \lambda\widetilde{\boldsymbol{M}}\boldsymbol{p}(t) + \frac{1-\lambda}{N}\boldsymbol{e} \qquad \xrightarrow{t\to\infty} \qquad \boldsymbol{r} = \lambda\widetilde{\boldsymbol{M}}\boldsymbol{r} + \frac{1-\lambda}{N}\boldsymbol{e}, \qquad \boldsymbol{e} = \begin{pmatrix}1\\ \vdots \\ 1\end{pmatrix} \in \mathbb{R}^N. \quad (19)$$

Reduction to a Pure Dominant Eigenvector Problem (The Google Matrix $\boldsymbol{G}$). Because $\boldsymbol{p}(t)$ and $\boldsymbol{r}$ are probability distributions whose entries sum to 1, we have the scalar identity $\boldsymbol{e}^T\boldsymbol{p}(t) = 1$ and $\boldsymbol{e}^T\boldsymbol{r} = 1$. Multiplying the constant teleportation term $\frac{1-\lambda}{N}\boldsymbol{e}$ by $1 = \boldsymbol{e}^T\boldsymbol{p}(t)$ (or $1 = \boldsymbol{e}^T\boldsymbol{r}$) allows us to factor $\boldsymbol{p}(t)$ and $\boldsymbol{r}$ on the right:

$$\lambda\widetilde{\boldsymbol{M}}\boldsymbol{r} + \frac{1-\lambda}{N}\boldsymbol{e} = \lambda\widetilde{\boldsymbol{M}}\boldsymbol{r} + \frac{1-\lambda}{N}\boldsymbol{e}\underbrace{(\boldsymbol{e}^T\boldsymbol{r})}_{=1} = \left(\lambda\widetilde{\boldsymbol{M}} + \frac{1-\lambda}{N}\boldsymbol{e}\boldsymbol{e}^T\right)\boldsymbol{r}.$$

Defining the **Google matrix** $\boldsymbol{G} \in \mathbb{R}^{N\times N}$ by

$$\boxed{\boldsymbol{G} = \lambda\widetilde{\boldsymbol{M}} + \frac{1-\lambda}{N}\boldsymbol{e}\boldsymbol{e}^T = \lambda\left(\boldsymbol{M} + \frac{1}{N}\boldsymbol{e}\boldsymbol{d}^T\right) + \frac{1-\lambda}{N}\boldsymbol{e}\boldsymbol{e}^T,} \quad (20)$$

the damped PageRank iteration and stationary equation become a **standard Markov chain and dominant eigenvector problem** for $\boldsymbol{G}$:

$$\boxed{\boldsymbol{p}(t+1) = \boldsymbol{G}\boldsymbol{p}(t) \qquad \xrightarrow{t\to\infty} \qquad \boldsymbol{G}\boldsymbol{r} = 1\cdot\boldsymbol{r}, \qquad \|\boldsymbol{r}\|_1 = \boldsymbol{e}^T\boldsymbol{r} = 1,\ \boldsymbol{r} > \boldsymbol{0}.} \quad (21)$$

Theorem 4.1 (Perron–Frobenius and Spectral Properties of the Google Matrix $\boldsymbol{G}$)**.** *For any directed graph $G = (V, E)$ and any damping factor $\lambda \in (0, 1)$, the Google matrix $\boldsymbol{G} = \lambda\widetilde{\boldsymbol{M}} + \frac{1-\lambda}{N}\boldsymbol{e}\boldsymbol{e}^T$ satisfies:*

1. ***Strictly positive Markov matrix:*** *Every entry of $\boldsymbol{G}$ is strictly positive ($G_{ij} \geq \frac{1-\lambda}{N} > 0$), and every column sums to 1:*

$$\boldsymbol{e}^T\boldsymbol{G} = \lambda\underbrace{\boldsymbol{e}^T\widetilde{\boldsymbol{M}}}_{=\boldsymbol{e}^T} + \frac{1-\lambda}{N}\underbrace{(\boldsymbol{e}^T\boldsymbol{e})}_{=N}\boldsymbol{e}^T = \lambda\boldsymbol{e}^T + (1-\lambda)\boldsymbol{e}^T = \boldsymbol{e}^T.$$

2. ***Unique dominant eigenvector:*** *By the* Perron–Frobenius Theorem *for strictly positive matrices, the spectral radius is $\rho(\boldsymbol{G}) = 1$, and $\mu_1 = 1$ is a **strictly simple** (algebraic and geometric multiplicity 1) dominant eigenvalue of $\boldsymbol{G}$. The corresponding normalized right eigenvector $\boldsymbol{r} = (PR(1), \ldots, PR(N))^T$ is **unique** and has strictly positive entries ($PR(x) \geq \frac{1-\lambda}{N} > 0$).*

3. ***Spectral gap and geometric convergence rate:*** *The second largest eigenvalue of $\boldsymbol{G}$ is bounded by the damping factor:*

$$|\mu_2(\boldsymbol{G})| \leq \lambda < 1.$$

Consequently, for any *initial probability vector $\boldsymbol{p}(0)$, the Power Method $\boldsymbol{p}(k) = \boldsymbol{G}^k\boldsymbol{p}(0)$ converges to the dominant eigenvector $\boldsymbol{r}$ at the geometric rate*

$$\|\boldsymbol{p}(k) - \boldsymbol{r}\|_1 \leq C\lambda^k.$$

For $\lambda = 0.85$, $\lambda^{50} = 0.85^{50} \approx 2.96 \times 10^{-4}$, so 50 matrix–vector products suffice for 3–4 digits of accuracy regardless of whether $N = 10$ or $N = 10^{10}$!

4. ***Matrix-free sparse evaluation:*** *Even though* $\boldsymbol{G}$ *is completely dense due to* $\boldsymbol{e}\boldsymbol{e}^T$ *and* $\boldsymbol{e}\boldsymbol{d}^T$*, each power-method step* $\boldsymbol{p}(k+1) = \boldsymbol{G}\boldsymbol{p}(k)$ *is computed using* (18) *as*

$$\boldsymbol{p}(k+1) = \lambda \boldsymbol{M}\boldsymbol{p}(k) + \frac{1 - \lambda + \lambda(\boldsymbol{d}^T \boldsymbol{p}(k))}{N}\boldsymbol{e},$$

which requires only a single sparse matrix–vector product $\boldsymbol{M}\boldsymbol{p}(k)$ *in* $O(|E|+N)$ *operations.*

Example 4.4 (Computed Dominant Eigenvector $\boldsymbol{r}$ on the 11-Node Wikimedia Graph)**.** Applying the damped PageRank power iteration ($\lambda = 0.85$, with sink node A teleporting uniformly to all $N = 11$ nodes) to the 11-node Wikimedia graph from Example 4.2 yields the dominant eigenvector $\boldsymbol{r}$ of $\boldsymbol{G}$ ($\boldsymbol{G}\boldsymbol{r} = \boldsymbol{r}$, expressed as percentages):

Node x	B	C	E	D	F	A	$G_1, \ldots, G_5$ (each)
In-degree	7	1	6	1	1	1	0
$r_x = PR(x)$	**38.4**%	**34.3**%	8.1%	3.9%	3.9%	3.3%	1.6%

Notice how the dominant eigenvector $\boldsymbol{r}$ captures *recursive importance* rather than simple in-degree:

- Node E has 6 incoming links (from F and $G_1, \ldots, G_5$), yet its PageRank is only $r_E = 8.1\%$ because $G_1, \ldots, G_5$ have no incoming links of their own (their PageRank is purely the teleportation floor $\frac{1-0.85}{11} + 0.85\frac{PR(A)}{11} \approx 1.62\%$).
- Node C has only **one** incoming link, but that link comes from the most important node B ($r_B = 38.4\%$), which has no other outgoing links ($\text{out}(B) = 1$). Consequently, C inherits $\lambda \cdot r_B = 0.85 \times 38.4\%$ of B's rank, giving $r_C = 34.3\%$!

Personalized PageRank (Recommender Systems & Topic-Sensitive Ranking). Instead of teleporting uniformly to all nodes in the network via $\frac{1}{N}\boldsymbol{e}$, we can bias random jumps toward a specific subset of "seed" or "context" nodes $\mathcal{S} \subset V$ (e.g., books, movies, or songs highly rated by a user in a recommender system, articles in a specific scientific field, a user's bookmarked pages, or proteins known to be associated with a disease). We encode this seed set via a vector of non-negative **preference weights** $\boldsymbol{c} \in \mathbb{R}^N$ supported on $\mathcal{S}$ ($c_i > 0$ for $i \in \mathcal{S}$, such as user ratings or binary indicators $c_i = 1$, and $c_i = 0$ for $i \notin \mathcal{S}$), normalized into the **personalization probability vector** $\tilde{\boldsymbol{c}} = \frac{\boldsymbol{c}}{\boldsymbol{e}^T\boldsymbol{c}}$ ($\tilde{c}_i \geq 0$, $\boldsymbol{e}^T\tilde{\boldsymbol{c}} = 1$). Replacing the uniform vector $\frac{1}{N}\boldsymbol{e}$ by $\tilde{\boldsymbol{c}}$ yields:

$$\boldsymbol{r} = \lambda\widetilde{\boldsymbol{M}}\boldsymbol{r} + (1-\lambda)\tilde{\boldsymbol{c}} \qquad \Longleftrightarrow \qquad \boxed{\widetilde{\boldsymbol{G}}\,\boldsymbol{r} = \boldsymbol{r}, \qquad \widetilde{\boldsymbol{G}} = \lambda\widetilde{\boldsymbol{M}} + (1-\lambda)\tilde{\boldsymbol{c}}\,\boldsymbol{e}^T.} \tag{22}$$

Because $\boldsymbol{e}^T\widetilde{\boldsymbol{G}} = \lambda\boldsymbol{e}^T + (1-\lambda)(\boldsymbol{e}^T\tilde{\boldsymbol{c}})\boldsymbol{e}^T = \boldsymbol{e}^T$, the personalized Google matrix $\widetilde{\boldsymbol{G}}$ remains column-stochastic with dominant eigenvalue $\mu_1 = 1$, and its dominant eigenvector $\boldsymbol{r}$ measures node importance *relative to the seed set* $\mathcal{S}$ *and preference weights* $\boldsymbol{c}$.

5 Sparse Direct Solvers and Matrix Reorderings

5.1 Block LU Factorization, Fill-In, and Elimination Graphs

Consider the first step of Gaussian elimination (LU factorization) on a sparse matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ partitioned into its top-left pivot $\alpha = a_{11} \neq 0$, first row $\boldsymbol{r} \in \mathbb{R}^{1\times(n-1)}$, first column $\boldsymbol{c} \in \mathbb{R}^{(n-1)\times 1}$,

and trailing submatrix $\boldsymbol{B} \in \mathbb{R}^{(n-1)\times(n-1)}$:

$$\boldsymbol{A} = \begin{pmatrix} \alpha & \boldsymbol{r} \\ \boldsymbol{c} & \boldsymbol{B} \end{pmatrix} = \begin{pmatrix} 1 & \boldsymbol{0} \\ \boldsymbol{c}/\alpha & \boldsymbol{I} \end{pmatrix} \begin{pmatrix} \alpha & \boldsymbol{r} \\ \boldsymbol{0} & \boldsymbol{C} \end{pmatrix} = \boldsymbol{LU}, \tag{23}$$

where the remaining $(n-1) \times (n-1)$ block to be factored is the **Schur complement**:

$$\boldsymbol{C} = \boldsymbol{B} - \frac{1}{\alpha}\boldsymbol{cr}. \tag{24}$$

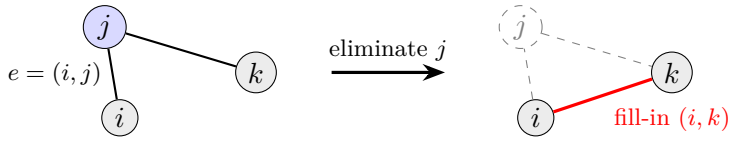
If $\alpha = 0$ (or is very small), row permutations $\boldsymbol{P}_r\boldsymbol{A} = \boldsymbol{LU}$ (pivoting) are required for numerical stability.

Fill-in: Notice that the rank-1 outer product $\boldsymbol{cr}/\alpha$ modifies entry (i,k) whenever $c_i = a_{i1} \neq 0$ and $r_k = a_{1k} \neq 0$. More generally, when eliminating pivot row/column j:

$$\begin{pmatrix} a_{jj} & a_{jk} \\ a_{ij} & 0 \end{pmatrix} \xrightarrow{\text{eliminate } j} \begin{pmatrix} a_{jj} & a_{jk} \\ 0 & F = \tilde{a}_{ik} = -\frac{a_{ij}a_{jk}}{a_{jj}} \neq 0 \end{pmatrix}.$$

A previously zero entry $a_{ik} = 0$ becomes a non-zero entry $F \neq 0$; this new non-zero is called **fill-in**.

Graph Interpretation: Associate with the sparse symmetric matrix $\boldsymbol{A}$ an undirected *adjacency graph* $G = (V, E)$, where vertex $i \in \{1, \ldots, n\}$ corresponds to row/column i, and an edge $(i,j) \in E$ exists if and only if $a_{ij} \neq 0$.



Before eliminating j New edge (i,k) added

Elimination Rule: When vertex j is eliminated, vertex j and its incident edges are removed, and **new edges (fill-in) are added between all pairs of neighbors of j that were not already connected** (i.e., the neighbors of j form a clique).

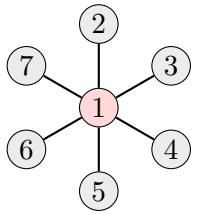
Example 5.1 (Fill-In in the 2D Poisson Matrix $\boldsymbol{K}_{2\mathrm{D}}$)**.** For the 2D Poisson problem on an $N \times N$ grid ordered row-by-row $(1,2,3,4 \mid 5,6,7,8 \mid 9,10,11,12\ldots)$, the 5-point stencil produces a block tridiagonal matrix:

$$\boldsymbol{K}_{2\mathrm{D}} = \begin{pmatrix} \boldsymbol{K}+2\boldsymbol{I} & -\boldsymbol{I} & & \\ -\boldsymbol{I} & \boldsymbol{K}+2\boldsymbol{I} & -\boldsymbol{I} & \\ & \ddots & \ddots & \ddots \\ & & -\boldsymbol{I} & \boldsymbol{K}+2\boldsymbol{I} \end{pmatrix}, \qquad \boldsymbol{K} = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \end{pmatrix}.$$

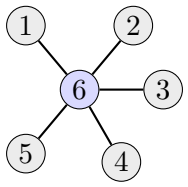
Between the main tridiagonal band and the outer $-\boldsymbol{I}$ bands (which are at distance N from the diagonal), all entries are initially zero. However, during LU factorization, eliminating row 1 creates fill-in F between its right neighbor (2) and its top grid neighbor $(N+1)$, which then cascades across the entire band between distance 1 and distance N:

$$\begin{pmatrix} 4 & -1 & & & -1 & & & \\ -1 & 4 & -1 & & F & -1 & & \\ & -1 & 4 & -1 & F & F & -1 & \\ & & -1 & 4 & \cdots & \cdots & \cdots & -1 \\ -1 & F & F & \cdots & 4 & -1 & & \end{pmatrix}.$$

Example 5.2 (Star Graph: Arrowhead vs. Hub-Last Ordering)**.** Consider a star graph where a central hub node is connected to leaf nodes:



(a) Hub numbered first (1)



(b) Hub numbered last (6)

- In ordering (a), where the hub is numbered 1, the first row and column are completely dense:

$$\boldsymbol{A}_1 = \begin{pmatrix} \times & \times & \times & \times & \times & \times \\ \times & \times & 0 & 0 & 0 & 0 \\ \times & 0 & \times & 0 & 0 & 0 \\ \times & 0 & 0 & \times & 0 & 0 \\ \times & 0 & 0 & 0 & \times & 0 \\ \times & 0 & 0 & 0 & 0 & \times \end{pmatrix}.$$

  Eliminating node 1 connects all of its neighbors $(2, \ldots, 6)$, turning the remaining submatrix into a completely dense clique (**100**% fill-in)!

- In ordering (b), obtained by symmetrically permuting rows and columns via $\boldsymbol{PAP}^T$ so that the leaves are numbered $1, \ldots, 5$ and the hub is numbered last (6), the matrix has an upward-pointing arrowhead structure:

$$\boldsymbol{PAP}^T = \begin{pmatrix} \times & 0 & 0 & 0 & 0 & \times \\ 0 & \times & 0 & 0 & 0 & \times \\ 0 & 0 & \times & 0 & 0 & \times \\ 0 & 0 & 0 & \times & 0 & \times \\ 0 & 0 & 0 & 0 & \times & \times \\ \times & \times & \times & \times & \times & \times \end{pmatrix} \xrightarrow{\text{LU}} \textbf{zero fill-in!}$$

  Because each leaf $1, \ldots, 5$ has only a single neighbor (node 6), eliminating them introduces no new edges at all.

5.2 Reordering Algorithms

To minimize fill-in and factorization cost in sparse direct solvers, we seek a permutation matrix $\boldsymbol{P}$ and solve $(\boldsymbol{PAP}^T)(\boldsymbol{Px}) = \boldsymbol{Pb}$.

1. **Red-Black (Checkerboard) Ordering:** Color the nodes of a 2D grid like a checkerboard into two independent sets: Red (R) and Black (B), where no two nodes of the same color share an edge (every Red node only connects to Black neighbors, and vice versa). Numbering all Red nodes first $(1, 2, 3, 4)$ and all Black nodes second $(5, 6, 7, 8)$ permutes the 2D Poisson matrix into a 2×2 block form:

$$\boldsymbol{PK}_{\text{2D}}\boldsymbol{P}^T = \begin{pmatrix} 4\boldsymbol{I}_R & \boldsymbol{C} \\ \boldsymbol{C}^T & 4\boldsymbol{I}_B \end{pmatrix}.$$

   Since the top-left block $4\boldsymbol{I}_R$ is strictly diagonal, all Red unknowns can be eliminated in parallel in a single step!

2. **Minimum Degree (MD / AMD / QMD):** A greedy local heuristic inspired by the star-graph example:
   - At each step, select and eliminate the vertex with the **smallest current degree** (fewest neighbors $\implies$ smallest clique of fill-in edges added).
   - Update the elimination graph (remove the vertex, connect its neighbors into a clique) and update vertex degrees.
   - Practical implementations use *Quotient Minimum Degree (QMD)* or *Approximate Minimum Degree (AMD)* to update degrees without explicitly forming all fill-in edges.
3. **Nested Dissection (ND):** A global divide-and-conquer strategy:
   - Find a small vertex **separator** S that splits the graph into two disconnected subgraphs P and Q of roughly equal size (no direct edges between P and Q).
   - Number the nodes in P first, then the nodes in Q, and place the separator nodes S **last**:
   $$\boldsymbol{PAP}^T = \begin{pmatrix} \boldsymbol{A}_{PP} & \boldsymbol{O} & \boldsymbol{A}_{PS} \\ \boldsymbol{O} & \boldsymbol{A}_{QQ} & \boldsymbol{A}_{QS} \\ \boldsymbol{A}_{SP} & \boldsymbol{A}_{SQ} & \boldsymbol{A}_{SS} \end{pmatrix}.$$
   - Because of the $\boldsymbol{O}$ blocks, eliminating P and Q produces zero fill-in between P and Q and can be done completely in parallel. Applying this recursively to P and Q yields optimal asymptotic complexity for 2D/3D mesh problems.
4. **(Reverse) Cuthill–McKee (CM / RCM):** Designed to reduce the **bandwidth** of $\boldsymbol{A}$ (clustering all non-zeros close to the main diagonal):
   (a) Pick a starting vertex with minimum degree (or a peripheral node).
   (b) Perform **Breadth-First Search (BFS)** level by level.
   (c) Within each level, sort the newly visited neighbors by **increasing degree**.
   (d) **RCM:** Reverse the resulting ordering at the end (which further reduces profile/envelope fill-in when high-degree nodes are pushed toward the end).

6 Quadratic Function Minimization and GMRES

6.1 Quadratic Cost Function and Level Sets

When $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ is symmetric positive definite ($\boldsymbol{A} = \boldsymbol{A}^T \succ 0$), solving $\boldsymbol{Ax} = \boldsymbol{b}$ is equivalent to minimizing the strictly convex quadratic function

$$f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\boldsymbol{Ax} - \boldsymbol{x}^T\boldsymbol{b}. \tag{25}$$

Let $\boldsymbol{x}^* = \boldsymbol{A}^{-1}\boldsymbol{b}$ be the exact solution ($\boldsymbol{Ax}^* - \boldsymbol{b} = \boldsymbol{0}$), and let $\boldsymbol{y} = \boldsymbol{x} - \boldsymbol{x}^*$ ($\iff$ $\boldsymbol{x} = \boldsymbol{y} + \boldsymbol{x}^*$) be the error vector. Expanding $f(\boldsymbol{y} + \boldsymbol{x}^*)$ around $\boldsymbol{x}^*$ gives:

$$\begin{aligned} f(\boldsymbol{y} + \boldsymbol{x}^*) &= \frac{1}{2}(\boldsymbol{y} + \boldsymbol{x}^*)^T\boldsymbol{A}(\boldsymbol{y} + \boldsymbol{x}^*) - (\boldsymbol{y} + \boldsymbol{x}^*)^T\boldsymbol{b} \\ &= \frac{1}{2}\boldsymbol{y}^T\boldsymbol{Ay} + \boldsymbol{y}^T\underbrace{(\boldsymbol{Ax}^* - \boldsymbol{b})}_{\boldsymbol{0}} + \underbrace{\left[\frac{1}{2}(\boldsymbol{x}^*)^T\boldsymbol{Ax}^* - (\boldsymbol{x}^*)^T\boldsymbol{b}\right]}_{f(\boldsymbol{x}^*)} \\ &= \frac{1}{2}\boldsymbol{y}^T\boldsymbol{Ay} + f(\boldsymbol{x}^*). \end{aligned}$$

Now consider a **level set** (contour surface) $f(\boldsymbol{x}) = c$ for some constant $c > f(\boldsymbol{x}^*)$:

$$f(\boldsymbol{x}) = c \quad \Longleftrightarrow \quad \frac{1}{2}\boldsymbol{y}^T\boldsymbol{A}\boldsymbol{y} = c - f(\boldsymbol{x}^*) =: k \quad \Longleftrightarrow \quad \boldsymbol{y}^T\boldsymbol{A}\boldsymbol{y} = 2k.$$

Using the orthogonal spectral decomposition $\boldsymbol{A} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T$ and changing coordinates to $\boldsymbol{z} = \boldsymbol{V}^T\boldsymbol{y}$ (aligned with the eigenvectors of $\boldsymbol{A}$), the level set equation becomes:

$$\boldsymbol{y}^T\boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T\boldsymbol{y} = \boldsymbol{z}^T\boldsymbol{\Lambda}\boldsymbol{z} = 2k.$$

In 2D ($\boldsymbol{z} = (z_1, z_2)^T$), this is the equation of an **ellipse**:

$$\lambda_1 z_1^2 + \lambda_2 z_2^2 = 2k \quad \Longleftrightarrow \quad \underbrace{\frac{z_1^2}{(2k/\lambda_1)}}_{\alpha^2} + \underbrace{\frac{z_2^2}{(2k/\lambda_2)}}_{\beta^2} = 1, \tag{26}$$

with semi-axes of lengths $\alpha = \sqrt{\frac{2k}{\lambda_1}}$ and $\beta = \sqrt{\frac{2k}{\lambda_2}}$ directed along the eigenvectors of $\boldsymbol{A}$.

- Larger eigenvalues λ_i correspond to *shorter* semi-axes $\sqrt{2k/\lambda_i}$ (steeper curvature).
- The ratio of the longest to the shortest semi-axis of the contour ellipsoid is $\sqrt{\lambda_{\max}/\lambda_{\min}} = \sqrt{\kappa_2(\boldsymbol{A})}$. When $\kappa_2(\boldsymbol{A}) \gg 1$, the level sets are highly elongated valleys, making gradient descent methods zig-zag and converge slowly.

6.2 Krylov Subspace Minimization and the GMRES Idea

Now let $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ be a general (possibly non-symmetric) invertible matrix, and let $\bar{\boldsymbol{x}} = \boldsymbol{A}^{-1}\boldsymbol{b}$ denote the exact solution of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$. At step k, let $\boldsymbol{e}_k = \boldsymbol{x}_k - \bar{\boldsymbol{x}}$ be the error and $\boldsymbol{r}_k = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}_k = -\boldsymbol{A}\boldsymbol{e}_k$ be the residual. Notice that minimizing the Euclidean norm of the residual $\|\boldsymbol{r}_k\|_2$ is equivalent to minimizing the $\boldsymbol{A}^T\boldsymbol{A}$-norm of the error:

$$\|\boldsymbol{e}_k\|^2_{\boldsymbol{A}^T\boldsymbol{A}} = \|\boldsymbol{x}_k - \bar{\boldsymbol{x}}\|^2_{\boldsymbol{A}^T\boldsymbol{A}} = \boldsymbol{e}_k^T\boldsymbol{A}^T\boldsymbol{A}\boldsymbol{e}_k = (\boldsymbol{A}\boldsymbol{e}_k)^T(\boldsymbol{A}\boldsymbol{e}_k) = (\boldsymbol{A}\boldsymbol{x}_k - \boldsymbol{b})^T(\boldsymbol{A}\boldsymbol{x}_k - \boldsymbol{b}) = \|\boldsymbol{r}_k\|_2^2.$$

The **Generalized Minimal Residual method (GMRES)** finds the iterate $\boldsymbol{x}_k$ in the affine Krylov subspace $\boldsymbol{x}_0 + \mathcal{K}_k(\boldsymbol{A}, \boldsymbol{r}_0)$ that minimizes the residual 2-norm:

$$\boldsymbol{x}_k = \underset{\boldsymbol{x}\in\boldsymbol{x}_0+\mathcal{K}_k(\boldsymbol{A},\boldsymbol{r}_0)}{\arg\min} \|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}\|_2, \qquad \mathcal{K}_k(\boldsymbol{A}, \boldsymbol{r}_0) = \operatorname{span}\{\boldsymbol{r}_0, \boldsymbol{A}\boldsymbol{r}_0, \boldsymbol{A}^2\boldsymbol{r}_0, \ldots, \boldsymbol{A}^{k-1}\boldsymbol{r}_0\}. \tag{27}$$

Let $\boldsymbol{K}_k = (\boldsymbol{r}_0, \boldsymbol{A}\boldsymbol{r}_0, \boldsymbol{A}^2\boldsymbol{r}_0, \ldots, \boldsymbol{A}^{k-1}\boldsymbol{r}_0) \in \mathbb{R}^{n\times k}$ be the Krylov matrix. For $\boldsymbol{x}_0 = \boldsymbol{0}$, any $\boldsymbol{x}_k \in \mathcal{K}_k(\boldsymbol{A}, \boldsymbol{r}_0)$ could be written as $\boldsymbol{x}_k = \boldsymbol{K}_k\boldsymbol{c}$, turning the problem into $\min_{\boldsymbol{c}} \|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{K}_k\boldsymbol{c}\|_2$. However, the columns $\boldsymbol{A}^j\boldsymbol{r}_0$ rapidly align with the dominant eigenvector of $\boldsymbol{A}$ (power iteration!), making $\boldsymbol{K}_k$ severely ill-conditioned.

Instead, GMRES builds an **orthonormal basis** $\boldsymbol{V}_k = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k)$ (analogous to the $\boldsymbol{Q}_k$ factor in $\boldsymbol{K}_k = \boldsymbol{Q}_k\boldsymbol{R}_k$) of $\mathcal{K}_k(\boldsymbol{A}, \boldsymbol{r}_0)$ step by step using Gram–Schmidt orthogonalization:

$$\begin{aligned}
\boldsymbol{v}_1 &= \frac{\boldsymbol{r}_0}{\|\boldsymbol{r}_0\|_2}, \\
\boldsymbol{v}_2 &= \frac{\boldsymbol{A}\boldsymbol{v}_1 - ((\boldsymbol{A}\boldsymbol{v}_1)^T\boldsymbol{v}_1)\boldsymbol{v}_1}{\|\boldsymbol{A}\boldsymbol{v}_1 - ((\boldsymbol{A}\boldsymbol{v}_1)^T\boldsymbol{v}_1)\boldsymbol{v}_1\|_2}, \\
\boldsymbol{v}_3 &= \frac{\boldsymbol{A}\boldsymbol{v}_2 - ((\boldsymbol{A}\boldsymbol{v}_2)^T\boldsymbol{v}_1)\boldsymbol{v}_1 - ((\boldsymbol{A}\boldsymbol{v}_2)^T\boldsymbol{v}_2)\boldsymbol{v}_2}{\|\boldsymbol{A}\boldsymbol{v}_2 - ((\boldsymbol{A}\boldsymbol{v}_2)^T\boldsymbol{v}_1)\boldsymbol{v}_1 - ((\boldsymbol{A}\boldsymbol{v}_2)^T\boldsymbol{v}_2)\boldsymbol{v}_2\|_2}, \quad \ldots
\end{aligned}$$

6.3 Arnoldi Orthogonalization and the Hessenberg Decomposition

Formalizing this Gram–Schmidt process on the Krylov subspace yields **Arnoldi's algorithm** (written in Modified Gram–Schmidt form for numerical stability):

Algorithm 1: Arnoldi Orthogonalization

```
1 v_1 = r_0/‖r_0‖_2
2 for j = 1, ..., m do
3 |   w = A v_j
4 |   for k = 1, ..., j do
5 |   |   h_{k,j} = w^T v_k
6 |   |   w = w − h_{k,j} v_k
7 |   h_{j+1,j} = ‖w‖_2
8 |   v_{j+1} = w/h_{j+1,j}
```

From lines 7–8 of the Arnoldi algorithm, at step j we have

$$\boldsymbol{w} = h_{j+1,j}\boldsymbol{v}_{j+1} = \boldsymbol{A}\boldsymbol{v}_j - \sum_{k=1}^{j} h_{k,j}\boldsymbol{v}_k \quad \Longleftrightarrow \quad \boldsymbol{A}\boldsymbol{v}_j = \sum_{k=1}^{j+1} h_{k,j}\boldsymbol{v}_k.$$

Writing this relation in matrix form for $j = 1, \ldots, m$ with $\boldsymbol{V}_m = (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_m) \in \mathbb{R}^{n\times m}$ and $\boldsymbol{V}_{m+1} = (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{m+1}) \in \mathbb{R}^{n\times(m+1)}$ gives the **Arnoldi relation**:

$$\boldsymbol{A}\boldsymbol{V}_m = (\boldsymbol{A}\boldsymbol{v}_1,\ \boldsymbol{A}\boldsymbol{v}_2,\ \ldots,\ \boldsymbol{A}\boldsymbol{v}_m) = \left(\sum_{k=1}^{2} h_{k,1}\boldsymbol{v}_k,\ \sum_{k=1}^{3} h_{k,2}\boldsymbol{v}_k,\ \ldots,\ \sum_{k=1}^{m+1} h_{k,m}\boldsymbol{v}_k\right) = \boldsymbol{V}_{m+1}\widetilde{\boldsymbol{H}}_m, \tag{28}$$

where $\widetilde{\boldsymbol{H}}_m \in \mathbb{R}^{(m+1)\times m}$ is an **upper Hessenberg matrix**:

$$\widetilde{\boldsymbol{H}}_m = \begin{pmatrix} h_{1,1} & h_{1,2} & \cdots & h_{1,m} \\ h_{2,1} & h_{2,2} & \cdots & h_{2,m} \\ 0 & h_{3,2} & \ddots & \vdots \\ \vdots & \ddots & \ddots & h_{m,m} \\ 0 & \cdots & 0 & h_{m+1,m} \end{pmatrix}.$$

6.4 Reducing GMRES to a Small Least-Squares Problem

At iteration m (or $i+1$), any candidate solution $\boldsymbol{x}_m \in \boldsymbol{x}_0 + \mathcal{K}_m(\boldsymbol{A}, \boldsymbol{r}_0)$ can be written as

$$\boldsymbol{x}_m = \boldsymbol{x}_0 + \boldsymbol{V}_m\boldsymbol{y}, \qquad \boldsymbol{y} \in \mathbb{R}^m.$$

Using the Arnoldi relation $\boldsymbol{A}\boldsymbol{V}_m = \boldsymbol{V}_{m+1}\widetilde{\boldsymbol{H}}_m$ and the fact that $\boldsymbol{r}_0 = \|\boldsymbol{r}_0\|_2\boldsymbol{v}_1 = \boldsymbol{V}_{m+1}(\|\boldsymbol{r}_0\|_2\boldsymbol{e}_1)$ (where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^T \in \mathbb{R}^{m+1}$), the residual norm simplifies dramatically:

$$\begin{aligned} \|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}_m\|_2 &= \|\boldsymbol{b} - \boldsymbol{A}(\boldsymbol{x}_0 + \boldsymbol{V}_m\boldsymbol{y})\|_2 = \|\boldsymbol{r}_0 - \boldsymbol{A}\boldsymbol{V}_m\boldsymbol{y}\|_2 \\ &= \|\boldsymbol{r}_0 - \boldsymbol{V}_{m+1}\widetilde{\boldsymbol{H}}_m\boldsymbol{y}\|_2 = \left\|\boldsymbol{V}_{m+1}\left(\|\boldsymbol{r}_0\|_2\boldsymbol{e}_1 - \widetilde{\boldsymbol{H}}_m\boldsymbol{y}\right)\right\|_2. \end{aligned}$$

Because $\boldsymbol{V}_{m+1}$ has orthonormal columns ($\boldsymbol{V}_{m+1}^T\boldsymbol{V}_{m+1} = \boldsymbol{I}_{m+1}$), it preserves Euclidean norms for any vector $\boldsymbol{z} \in \mathbb{R}^{m+1}$:

$$\|\boldsymbol{V}_{m+1}\boldsymbol{z}\|_2^2 = \boldsymbol{z}^T\boldsymbol{V}_{m+1}^T\boldsymbol{V}_{m+1}\boldsymbol{z} = \boldsymbol{z}^T\boldsymbol{I}\boldsymbol{z} = \|\boldsymbol{z}\|_2^2.$$

Therefore, the n-dimensional minimization problem reduces to a tiny $(m+1) \times m$ least-squares problem:

$$\boxed{\|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}_m\|_2 = \left\| \|\boldsymbol{r}_0\|_2 \boldsymbol{e}_1 - \widetilde{\boldsymbol{H}}_m \boldsymbol{y} \right\|_2 .} \tag{29}$$

Since $m \ll n$, this small upper Hessenberg least-squares problem $\min_{\boldsymbol{y} \in \mathbb{R}^m} \|\boldsymbol{c} - \widetilde{\boldsymbol{H}}_m \boldsymbol{y}\|_2$ (with $\boldsymbol{c} = \|\boldsymbol{r}_0\|_2 \boldsymbol{e}_1$) is solved cheaply at each iteration using a QR factorization (via Givens rotations) $\boldsymbol{Q}\widetilde{\boldsymbol{H}}_m = \boldsymbol{R}$:

$$\|\boldsymbol{c} - \widetilde{\boldsymbol{H}}_m \boldsymbol{y}\|_2 = \|\boldsymbol{Q}(\boldsymbol{c} - \widetilde{\boldsymbol{H}}_m \boldsymbol{y})\|_2 = \|\boldsymbol{Q}\boldsymbol{c} - \boldsymbol{R}\boldsymbol{y}\|_2,$$

where the last component of $\boldsymbol{Q}\boldsymbol{c} = \|\boldsymbol{r}_0\|_2 \boldsymbol{Q}\boldsymbol{e}_1$ immediately gives the current residual norm $\|\boldsymbol{r}_m\|_2$ without even forming $\boldsymbol{x}_m$ until convergence!